

Complete documentation

Every TRUSTMESH document in one file: overview, architecture, contracts, IoT protocol, API, security, setup, hardware, demo, tests.

Generated 2026-09-29 from the repository · local-first hackathon prototype

Contents

1. README
2. Architecture
3. Smart contracts
4. IoT integration & device protocol
5. API guide
6. Database
7. Security
8. Local setup
9. Troubleshooting
10. Judge demo script
11. Evaluation mapping
12. Wiring
13. Wiring tables (generated)
14. Hardware BOM
15. Hardware coverage
16. Deployment
17. Acceptance matrix
18. Test report
19. Implementation status
20. API endpoint reference

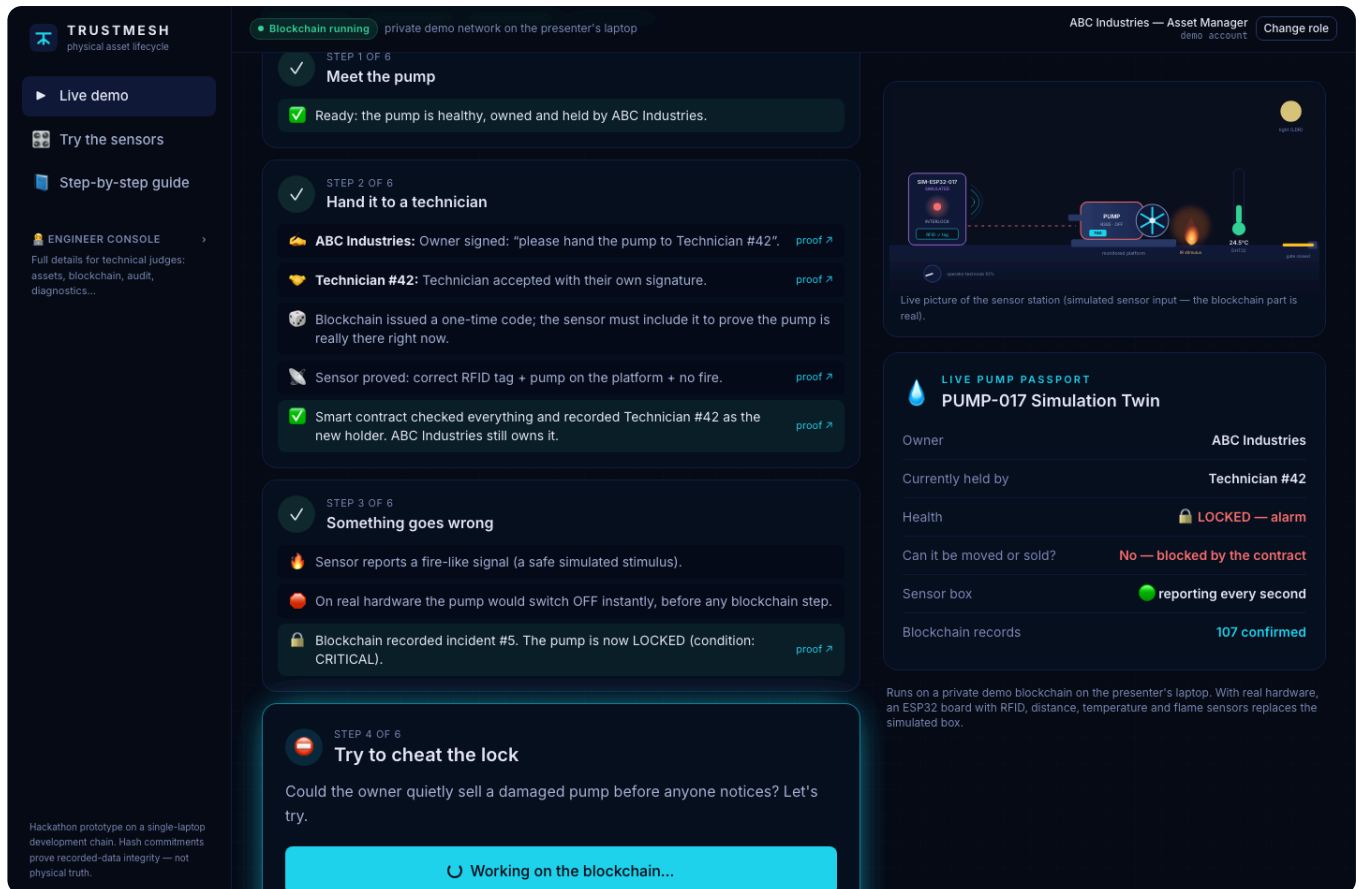
README.md

TRUSTMESH

Blockchain-verified physical asset lifecycle

Trust the asset, not the paperwork.

A sensor device watches a physical machine and signs what it sees. The evidence is hashed onto a blockchain, and smart contracts decide who may hold, sell, repair or release the machine.



Why

Physical assets such as pumps, generators and compressors change hands, break and get repaired. Today the records are paperwork anyone can edit, and nothing checks that the machine was actually present when someone signed.

TRUSTMESH connects the physical machine to rules that cannot be bypassed:

Physical asset → identified device → authenticated evidence → integrity hash
→ blockchain confirmation → contract-controlled decision → visible response

- **Custody handovers** complete only when the device proves, with a one-time challenge, that the tagged asset is really there.
- **Sensor alarms** (flame-like input, removal, overheating, ...) lock the asset on-chain. Normal readings or an "acknowledge" click cannot unlock it.
- **Repairs** need a credentialed technician plus a *different*, independent inspector before the asset is released.
- **Tampering** with any stored reading is detected against the on-chain hash.

Features

⚙️ 3 smart contracts	DeviceRegistry · EvidenceRegistry · AssetLifecycle : custody ≠ ownership, recipient-only acceptance, challenge-bound fresh evidence, REAL-vs-SIMULATED policy, incidents, credentials, independent inspection
🔑 Authenticated devices	Per-device HMAC-SHA256 over exact payload bytes, session and replay protection, duplicate detection, signed server responses and commands
📄 Verifiable evidence	Canonical JSON with fixed-point readings, hashed with keccak256 and committed on-chain. The integrity lab recomputes the hash in your browser

 Live dashboard	Trust-mesh map, 8-stage evidence pipeline, digital passports with QR codes and "stamps", role-aware workflows, built-in explorer for the local chain
 2-minute live demo	/demo : the whole story in 6 one-click steps with plain-English explanations and "proof" links to each transaction
 In-app guide	/guide : 8 hands-on missions, one-click identity switching and browser controls for a simulated device
 ESP32 firmware	6 hardware profiles covering 19 component types, pin maps generated from one file with a conflict checker, local safety interlocks that never wait for the blockchain
 Two transports	Wi-Fi (HTTP to the laptop's LAN IP) or USB serial bridge, using the same signed envelopes
 Reliability	Durable job queue with crash reconciliation, event indexer with a persistent position, chain-reset detection, SSE with polling fallback

Quick start

Requirements: Node.js 22.13+ or 24 (see `.nvmrc`), npm and Git. Python 3 and PlatformIO are needed only for firmware.

```
git clone https://github.com/<your-username>/trustmesh.git
cd trustmesh
npm ci
npm run setup      # creates .env, compiles contracts, generates ABIs and pin maps
npm run doctor     # checks Node, ports, chain and the LAN IPs to use for ESP32
npm run dev        # local chain → deploy → seed → API → web
```

Open <http://localhost:3000> — it lands on the **Live demo**: 6 big buttons that tell the whole story (hand-over → fire alarm → blocked sale → repair & independent inspection → tamper test), each a real blockchain transaction. No roles or setup needed.

Service	URL
Web app	http://localhost:3000
Guide	http://localhost:3000/guide
API + OpenAPI docs	http://localhost:4000/docs
Local EVM (Hardhat, chain 31337)	http://127.0.0.1:8545 (loopback only)

You don't need a wallet, faucet, cloud account or internet connection once the dependencies are installed. You sign in with built-in demo identities: an asset owner, Technician #42, Inspector #7, a platform admin and a buyer.

Scripted rehearsal (no hardware)

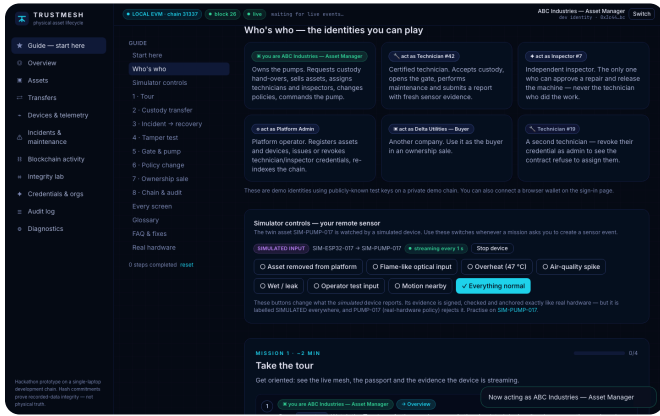
```
npm run demo:simulate -- --scenario full
```

This runs a simulated device through the **real** device protocol, backend checks and smart contracts:

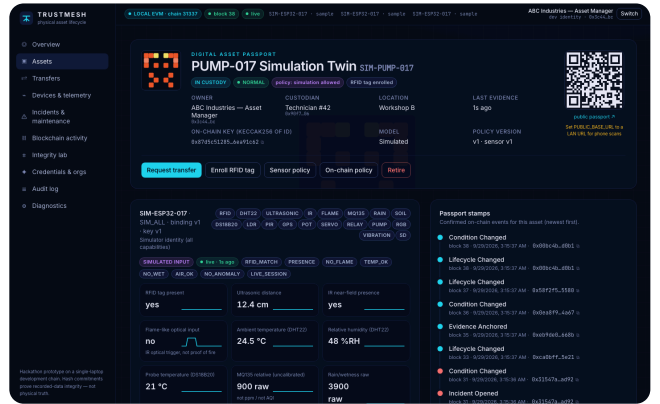
1. Custody transfer
2. Incident, with the transfer blocked
3. Maintenance
4. Self-inspection rejected
5. Independent approval and recovery
6. Tamper test

Screenshots

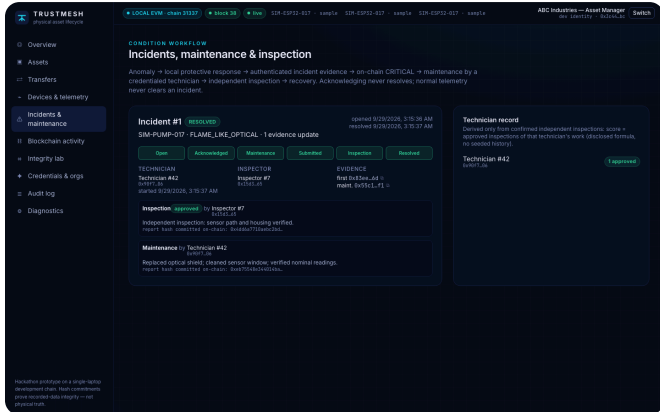
Guide



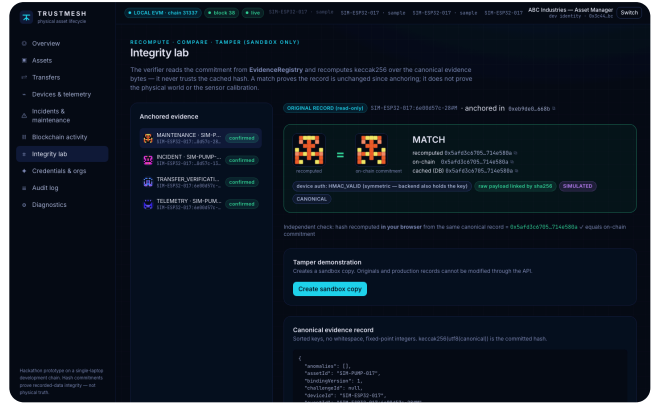
Asset passport



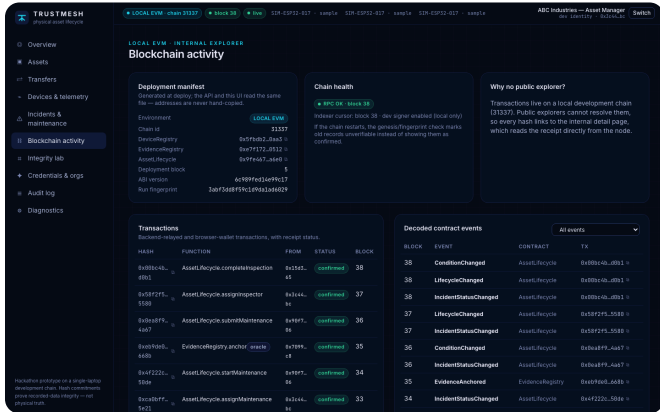
Incidents & inspection



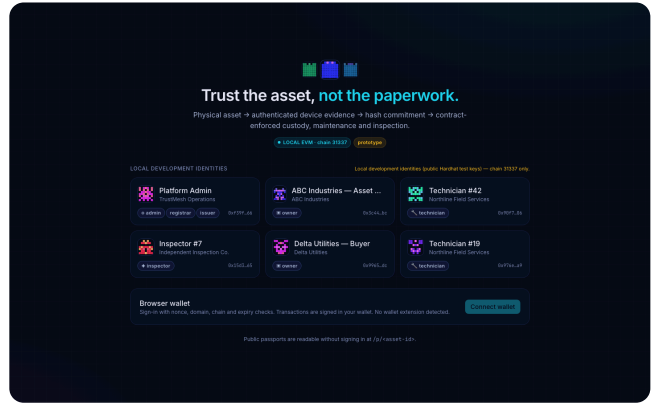
Integrity lab



Blockchain activity



Sign-in



Commands

Command	What it does
<code>npm run dev</code>	Starts everything in order and stops it all on Ctrl+C
<code>npm run demo:simulate [-- --scenario live custody incident integrity full]</code>	Simulated device, labelled SIMULATED everywhere
<code>npm run bridge -- --port <port> . -- --list</code>	USB serial bridge for a real ESP32
<code>npm run test:all</code>	Pin checks, shared protocol tests, contracts, API, end-to-end browser tests, firmware host tests
<code>npm run firmware:build</code>	Compiles all 6 ESP32 profiles
<code>npm run build</code>	Production builds (contracts, API bundle, web bundle)

Command	What it does
<code>npm run reset:demo -- --confirm</code>	Archives the database, redeploys and reseeds

Architecture

ESP32 firmware —Wi-Fi / USB bridge→ API (Fastify + SQLite) —oracle txs→ Local EVM
 (sense, interlock, sign) verify HMAC · policy · DeviceRegistry
 canonical evidence · outbox EvidenceRegistry
 Browser (React) ← SSE / REST → indexer ← contract events → AssetLifecycle
 demo identities → backend dev signer · wallets → sign transactions directly

apps/api	Fastify API, device protocol, policy engine, outbox, indexer, integrity lab
apps/web	React 19 + Vite + Tailwind 4 UI (includes /guide)
packages/contracts	Solidity contracts, Hardhat tests, deploy script → data/deployment.json
packages/shared	Protocol, canonical hashing, flags/capabilities, action map, ABIs, test vectors
firmware/	PlatformIO ESP32 firmware, pins/profiles.json (single source of truth)
tools/serial-bridge	USB serial ↔ API bridge
scripts/	setup, doctor, dev orchestrator, reset, simulator
tests/	end-to-end suite (isolated chain + API + Chromium), serial PTY test
docs/	full documentation

Hardware

The reference board is an **ESP32-DevKitC (WROOM-32)**.

Profile	Components
CORE	RC522 RFID, HC-SR04, IR, DHT22, flame sensor, servo gate, RGB LED
ENV	DHT22, MQ135, LDR, PIR, flame sensor, potentiometer, RGB LED
WET	Rain sensor, soil-moisture sensor, DS18B20, HC-SR04, IR, RGB LED
ACT	Relay → R385 pump, servo, vibration motor, NEO-6M GPS, microSD, potentiometer, RGB LED
FULL_A + FULL_B	Every component at once, split across two ESP32 boards

```
cd firmware
cp include/secrets.example.h include/secrets.h # device ID, secret, Wi-Fi, laptop LAN URL
pio run -e core -t upload --upload-port <port>
npm run bridge -- --port <port> # or use Wi-Fi with API_HOST=0.0.0.0
```

Have the Newrro ESP32-S3 kit? Follow the illustrated [hardware setup guide \(PDF\)](#), then use `npm run firmware:secrets` and `pio run -e newrro_full -t upload`.

Wiring, power and safety details: [docs/WIRING.md](#) · [Coverage of all 19 components](#) · [Bill of materials](#)

⚠ Use motors only with external supplies and low-voltage loads. No mains wiring. Use safe stimuli only, such as an IR remote for the flame sensor, never real fire.

Testing

Suite	Result
Smart contracts (Hardhat)	✅ 24/24
Shared protocol and hash vectors	✅ 4/4

Suite	Result
API (policy engine, outbox crash recovery)	✅ 10/10
End-to-end: isolated chain + API + UI in Chromium	✅ 24/24
Firmware host protocol tests, sharing test vectors with the backend	✅ 3/3
Firmware compile, all 6 profiles	✅ 6/6
Serial bridge over a pseudo-terminal	✅ 5/5

Details, environment and limitations: [TEST_REPORT.md](#)

Honest limitations

- **Prototype on a single-laptop development chain.** This is not a decentralized production network. The local chain resets when restarted, and old data is archived.
- **Hardware not yet physically tested.** The firmware compiles for every profile and the protocol is tested, but no real ESP32 or sensors have been wired up yet.
- **One trusted backend "oracle" interprets sensor readings.** Device authentication is symmetric (HMAC): the backend holds the key and could forge a message. This is not public-key device attestation.
- **A hash match proves the record is unchanged, not that the physical world matched it.** RFIDUIDs can be cloned. MQ135, LDR and soil readings are relative and uncalibrated.
- **Commands are acknowledged, not physically verified.** There is no flow sensor or gate-position sensor.
- **Demo identities use the public Hardhat test keys.** They only work on chain 31337, only from the host machine, and never in production mode.

Documentation

Topic	
How it fits together	Architecture · Smart contracts · IoT protocol
Running it	Local setup (macOS/Windows/Linux) · Troubleshooting · Deployment
Reference	API · Database · Security
Demo & judging	Judge demo script · Evaluation mapping · Acceptance matrix
Hardware	Wiring · Generated wiring tables · Coverage · BOM

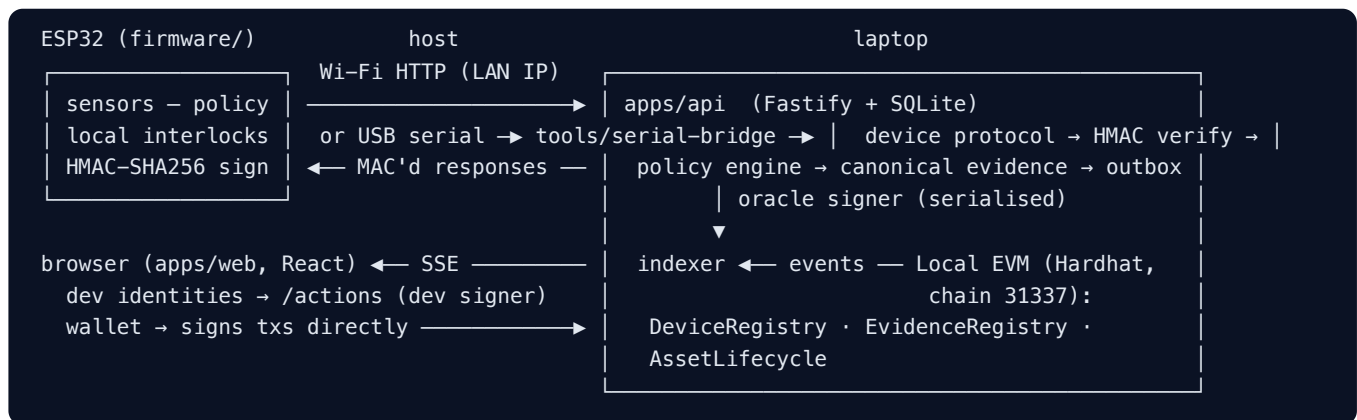
Tech stack

Solidity 0.8.28 · OpenZeppelin 5 · Hardhat 2 · viem · Fastify 5 · SQLite (`node:sqlite`) · React 19 · Vite 7 · Tailwind CSS 4 · Playwright · PlatformIO / Arduino-ESP32 · TypeScript throughout

License

No license has been chosen yet. Add a `LICENSE` file before accepting outside contributions. Third-party dependencies keep their own licenses.

Architecture



Components

Path	Role
<code>packages/contracts</code>	Solidity 0.8.28, OpenZeppelin 5 AccessControl/Pausable, Hardhat 2 + viem. Deploy script writes <code>data/deployment.json</code> (manifest).
<code>packages/shared</code>	Single source for flags, capability bits, readings catalogue, canonical JSON + keccak256 hashing, HMAC envelope (<code>node.ts</code>), <code>action→contract-call</code> map, generated ABIs, test vectors.
<code>apps/api</code>	Fastify API: device protocol, policy engine, evidence, durable outbox, indexer/projections, SSE, auth (dev identities + wallet sign-in), integrity lab, OpenAPI at <code>/docs</code> . SQLite via Node's built-in <code>node:sqlite</code> .
<code>apps/web</code>	React 19 + Vite + Tailwind 4 UI. Only relative <code>/api</code> paths (dev proxy).
<code>firmware</code>	PlatformIO/Arduino-ESP32. Control loop on core 1, network task on core 0. Pins generated from <code>firmware/pins/profiles.json</code> .
<code>tools/serial-bridge</code>	USB transport: forwards the firmware's request lines to the device API; never signs.
<code>scripts</code>	<code>setup</code> , <code>doctor</code> , <code>dev orchestrator</code> , <code>reset</code> , simulator + scenarios.

Source of truth

- **On-chain** (authoritative): ownership, custody, lifecycle, condition, transfers, challenges, incidents, credentials, device bindings/provenance, evidence commitments.
- **SQLite**: projections of chain state (rebuilt by the indexer from contract reads), plus off-chain operational data: raw authenticated payloads, canonical evidence bytes, sessions, commands/acks, reports, audit log, jobs.
- The UI shows projections only after the indexer has seen the confirmed event (`syncTo(block)`), and shows pending/failed pipeline stages explicitly.

Trust boundaries

1. **Device → backend**: HMAC-SHA256 with a per-device random 32-byte key. Symmetric: the backend could forge a device message. It is *not* public-key device attestation.
2. **Backend (oracle) → chain**: the oracle account (ORACLE_ROLE) anchors evidence and reports conditions. The contract does not verify HMACs; it verifies the device is registered, active and bound, takes provenance and binding version from `DeviceRegistry` (not from the oracle), enforces single-use event ids, challenge match, freshness, flags and REAL/SIMULATED policy. The oracle remains trusted for *physical interpretation* (readings → flags).
3. **Users → chain**: every lifecycle action is a transaction by the acting identity (dev key via the backend adapter, or the user's wallet). The contract enforces owner / recipient / technician / inspector permissions.

4. **Physical world:** RFID UUIDs can be cloned; sensors can be spoofed or mis-calibrated. A hash match proves record integrity, not physical truth.

A stronger future design: per-device secp256k1/P-256 keys in a secure element (ATECC608 / ESP32 flash encryption + secure boot), device-signed evidence verified on-chain (or via EIP-1271/precompile), and multiple independent oracles.

Data flow (custody)

owner `requestTransfer` → recipient `acceptTransfer` → indexer sees `TransferAccepted` → backend creates random challenge bound to {transfer, asset, recipient, policy version, device binding versions, deployment fingerprint} → oracle `issueChallenge` → device learns challenge from its authenticated poll → device telemetry includes `challengeId` → backend verifies MAC, session, freshness, binding, provenance, required flags → canonical evidence → `anchor` → `completeTransfer` (contract re-checks everything) → indexer → UI.

Chain reset handling

The Hardhat chain is ephemeral. `npm run dev` compares the chain's genesis hash and deployed bytecode with the manifest; if they differ it archives `data/trustmesh.db` to `data/archive/`, redeploys and reseeds (device secrets in `.local/devices.json` are kept). The API additionally refuses chain operations when the DB fingerprint ≠ manifest fingerprint, so old transactions are never shown as confirmed on a new chain.

Smart contracts

Solidity 0.8.28 (vialR, optimizer 200, cancun), OpenZeppelin 5.6.1. No upgradeability, no tokens, no escrow. ERC-721 was deliberately not used (a registry avoids transfer/approve bypass paths).

DeviceRegistry

`registerDevice(id, simulated, caps)`, `bindDevice(id, assetId)` (increments `bindingVersion`), `setCapabilities`, `rotateKey` (increments `keyVersion`), `revokeDevice`. REGISTRAR_ROLE only. `simulated` is immutable after registration.

EvidenceRegistry

`anchor({eventId, hash, deviceId, challengeId, observedAt, flags, policyVersion, kind})` — ORACLE_ROLE. Rejects zero/duplicate event ids, observation >120 s in the future, and unknown/revoked/unbound devices. Stores `assetId`, `bindingVersion`, `simulated` from **DeviceRegistry**. Kinds: 0 TELEMETRY, 1 TRANSFER_VERIFICATION, 2 INCIDENT, 3 MAINTENANCE. Not pausable (incident recording must never be suppressed).

AssetLifecycle — orthogonal state

- Lifecycle: REGISTERED → AVAILABLE / IN_CUSTODY (derived from owner≠custodian) · IN_TRANSIT (custody accepted, awaiting evidence) · UNDER_MAINTENANCE · UNDER_INSPECTION · RETIRED
- Condition: UNKNOWN · NORMAL · WARNING · CRITICAL · RECOVERY_PENDING
- Transfer: REQUESTED · ACCEPTED (ownership) · AWAITING_EVIDENCE (custody) · COMPLETED · CANCELLED · EXPIRED
- Incident: OPEN · ACKNOWLEDGED · MAINTENANCE_REQUIRED · MAINTENANCE_SUBMITTED · INSPECTION_PENDING · RESOLVED

Rules enforced

Rule	Where
Unique asset id; registrar-only registration	<code>registerAsset</code>
Owner-only activation, policy change (versioned; blocked during transfer), retirement	<code>activateAsset</code> , <code>setPolicy</code> , <code>retireAsset</code>
Owner initiates transfers; custody ≠ ownership	<code>requestTransfer</code> (<code>from</code> = custodian for CUSTODY, owner for OWNERSHIP)
Only designated recipient accepts; one active transfer; TTL 60 s–7 d; expiry/cancel	<code>acceptTransfer</code> , <code>cancelTransfer</code> , <code>expireTransfer</code>
Challenge committed on-chain by oracle; re-issue invalidates earlier evidence	<code>issueChallenge</code>
Custody completion: challenge match, kind, anchored after challenge, <code>observedAt</code> + <code>maxEvidenceAge</code> ≥ <code>now</code> , device active + same asset + same binding version, required flags, REAL-only if policy says so, single-use evidence	<code>_checkTransferEvidence</code>
Fresh condition check at completion (no CRITICAL / RECOVERY_PENDING / open incident / maintenance / retired)	<code>_requireOperable</code>
CRITICAL evidence opens one incident; repeats update it; NORMAL can't clear incident/CRITICAL	<code>reportCondition</code>
Acknowledge ≠ resolve	<code>acknowledgeIncident</code>

Rule	Where
Maintenance by valid (unexpired, unrevoked) TECHNICIAN credential; nominal MAINTENANCE evidence anchored after start	<code>assignMaintenance</code> , <code>startMaintenance</code> , <code>submitMaintenance</code>
Independent inspector (≠ technician) with valid INSPECTOR credential; approval is the only path back to NORMAL	<code>assignInspector</code> , <code>completeInspection</code>
Pause blocks registration and transfers, never incidents/evidence	<code>whenNotPaused</code> <code>placement</code>

Reputation input: `approvedInspections[technician]` increments only on an independent approval.

Policy

`Policy{requiredFlags, maxEvidenceAge, requireReal, version}` per asset (on-chain). Default custody policy: `RFID_MATCH | PRESENCE | NO_FLAME` , 120 s. Sensor *thresholds* (how readings become flags) are an off-chain, versioned, audited backend policy — see `apps/api/src/policy.ts` .

Tests

`npm run test:contracts` — 24 tests (see `TEST_REPORT.md`).

Deployment

`scripts/deploy.cjs` deploys the three contracts, grants `ORACLE_ROLE` to account #1, writes `data/deployment.json` (`chainId`, `addresses`, `abiVersion`, `deploymentBlock`, `genesisHash`, `fingerprint`). Backend and UI read only this manifest. `npm run dev` deploys automatically when needed. Addresses are deterministic on a fresh Hardhat chain but must never be hand-copied.

No MST (or other external network) integration existed or was added; public testnet deployment is described as an unverified extension in `DEPLOYMENT.md`.

IoT integration

Envelope (both transports)

```
{ "deviceId": "ESP32-017", "keyVersion": 1, "payloadB64": "<base64 of exact UTF-8 payload bytes>",  
  "macHex": "<hmac-sha256 hex>" }
```

MAC = HMAC-SHA256(key = 32-byte per-device secret, message = preimage):

```
domain (4 ASCII bytes: "TMD1" device→server, "TMS1" server→device)  
u8 len(deviceId) || deviceId (ASCII)  
u32be keyVersion  
u32be len(payload) || payload bytes
```

The backend verifies the MAC over the decoded bytes **before** parsing JSON; payloads > 4096 bytes are rejected. Server responses are sealed with `TMS1` and verified by the firmware (commands and challenges are therefore authenticated too). Shared vectors: `packages/shared/test-vectors.json`, checked by `npm run test:shared` and by the firmware host test `pio test -e native` (same C++ used on the ESP32).

This is **symmetric** authentication: the backend holds the key and could forge a message. It is not independent device attestation.

Payload v1 (authenticated)

field	meaning
<code>v</code>	schema version (only 1 accepted)
<code>type</code>	<code>hello · telemetry · poll · ack</code>
<code>deviceId</code> , <code>bootId</code>	identity; <code>bootId</code> random per boot
<code>sessionId</code>	from the authenticated hello response
<code>seq</code>	monotonically increasing for the boot
<code>eventId</code>	<code><bootId>--<seq></code> ; stable across retries
<code>uptimeMs</code> , <code>ts?</code>	device time; server receipt time is recorded separately
<code>bindingVersion</code> , <code>profile</code> , <code>caps</code>	declared configuration (binding checked against registry)
<code>r / q</code>	readings (explicit keys, see <code>READINGS</code> in <code>shared/constants.ts</code>) and per-reading quality (<code>ok</code> , <code>warmup</code> , <code>fault</code> , <code>absent</code> , <code>nofix</code> , <code>uncal</code>)
<code>challengeId?</code>	active custody challenge (from poll)
<code>local</code>	<code>pumpOn</code> , <code>interlock</code> , <code>manualStop</code> , <code>buffered</code> , <code>dropped</code>
<code>cmd</code>	(ack) `{commandId, status: applied`

Provenance (REAL/SIMULATED) is **never** read from the payload: it comes from the device's provisioning (DB + DeviceRegistry).

Sessions, replay, duplicates

- `hello` needs a random hex `nonce`; (device, `bootId`, `nonce`) is unique → replayed hello = `409 SESSION_REPLAY`. A new hello closes the previous session.

- Telemetry must reference a known session. Event id is unique per device: an identical retry returns the original (MAC'd) result with `duplicate:true` and creates nothing new; same id + different bytes = `409 CONFLICTING_DUPLICATE`; same `(session, seq)` with another id = `409 SEQ_REPLAY`.
- Samples from a closed session, out of order, or >30 s old are stored as **delayed** and never satisfy a challenge.
- A payload `assetId` that differs from the registered binding → `409 BINDING_MISMATCH`; stale `bindingVersion` → `409 BINDING_VERSION_STALE` (device re-hellos). Rebinding invalidates outstanding challenges and closes sessions.
- Rate limit: 15 req/s sustained, burst 30 per device. Body limit 8 KB.

Canonical evidence (what gets hashed)

Schema `trustmesh.evidence/1`: `{schema, eventId, kind, deviceId, assetId, bindingVersion, provenance, profile, sessionId, seq, observedAt, policyVersion, sensorPolicyVersion, challengeId, flags, anomalies[], payloadSha256, readings{key:{v,unit,q}}}`. Readings are fixed-point integers ($^{\circ}\text{C}\times 100$, mm, raw ADC, degrees $\times 1\text{e}7$...). Canonical bytes = JSON with recursively sorted keys, no whitespace, floats rejected. Hash = `keccak256(utf8(canonical))`; the hash and receipts are not part of the preimage. The raw RFID UID never enters canonical evidence — only `tagref = sha256("trustmesh-tag:"+UID)[0:16]`.

Transports

- **Wi-Fi**: firmware POSTs to `TM_BACKEND_URL` (laptop LAN IP, never localhost). API must listen on `0.0.0.0` (`API_HOST=0.0.0.0` in `.env`).
- **USB serial**: firmware prints `{"t":"req","id":N,"path":"/api/v1/device/...", "body":{envelope}}` lines; `npm run bridge -- --port <port>` POSTs them and writes `{"t":"res","id":N,"status":S,"body":{...}}`. Log lines start with `#`. The bridge announces itself with `{"t":"bridge","v":1}` every 2 s, forwards only the four device paths, and relays `!`... diagnostic lines you type.

Commands

Delivered in the MAC'd poll response: `{commandId, action, params, deviceId, assetId, bindingVersion, issuedAt, expiresAt (server s), txRef}`; TTL 60 s. Firmware re-validates device/asset/binding/expiry (server-time offset from each authenticated response — no NTP needed), dedupes the last 16 ids in NVS, marks a command `pending` in NVS before physical action and acks `uncertain` after a reset mid-action (never repeats it). Actions: `GATE_OPEN` (custodian only, references the custody tx), `GATE_CLOSE`, `PUMP_ON` (1–30 s, requires on-chain NORMAL/no incident/AVAILABLE|IN_CUSTODY), `PUMP_OFF` (always allowed), `INDICATE`, `CLEAR_LOCAL_STOP`. UI shows created → delivered → acknowledged separately; acknowledgement is not observed physical effect.

Provision / bind / revoke / rotate

UI → Devices (admin) or `POST /api/v1/devices` → secret shown once → put in `firmware/include/secrets.h`. Seeded devices' secrets live in `.local/devices.json` (git-ignored, kept across demo resets). `POST /devices/:id/bind` (new binding version), `/revoke` (on-chain + key retired), `/rotate-key` (new secret + on-chain keyVersion).

Simulator

`npm run demo:simulate` uses the SIMULATED identity `SIM-ESP32-017` bound to `SIM-PUMP-017` through the same HTTP protocol. Scenarios: `live` (interactive keys), `custody`, `incident`, `integrity`, `full`. Real-hardware policies reject its evidence (tested).

API

Base: `http://localhost:4000` (browser uses relative `/api/...` through the Vite proxy). Live OpenAPI UI: `/docs` ;
exported spec: [openapi.json](#). All errors: `{ "error": CODE, "message": text }`.

Auth: session cookie `tm_sid` (HttpOnly, SameSite=Strict, 12 h). Every non-GET session request (except `/auth/*` and `/device/*`) must send `x-csrf-token` (returned by `/auth/me` or login). Device endpoints use HMAC envelopes only.

Method & path	Who	Purpose
GET <code>/health</code> · <code>/ready</code>	public	liveness · readiness (503 when chain/manifest/fingerprint not OK)
GET <code>/api/v1/auth/personas</code>	public	local dev identities (only when dev signer enabled)
POST <code>/api/v1/auth/dev-login</code> <code>{persona}</code>	public (dev only)	session for a dev identity
POST <code>/api/v1/auth/nonce</code> <code>{address}</code> → <code>{message}</code>	public	SIWE-style message (domain, chain id, nonce, 5 min expiry)
POST <code>/api/v1/auth/wallet</code> <code>{message, signature}</code>	public	verifies domain, chain, expiry, single-use nonce, signature
GET <code>/api/v1/auth/me</code> · POST <code>/auth/logout</code>	session	
POST <code>/api/v1/device/{hello,telemetry,</code> <code>poll,ack}</code>	device (HMAC)	protocol — see IOT_INTEGRATION.md
GET <code>/api/v1/stream</code>	public	SSE (<code>Last-Event-ID</code> or <code>?last=</code> replay; <code>resync</code> after server restart)
GET <code>/api/v1/chain/manifest</code> · <code>/chain/status</code>	public	deployment manifest · RPC/indexer health
GET <code>/api/v1/chain/txs</code> · <code>/chain/events</code> · <code>/chain/tx/:hash</code>	session	local explorer (receipt read from node)
POST <code>/api/v1/chain/track</code> <code>{hash}</code>	session	record a browser-wallet tx sent by the signed-in wallet
POST <code>/api/v1/chain/reindex</code>	admin	reset indexer cursor
GET <code>/api/v1/overview</code>	session	dashboard counts
GET/POST <code>/api/v1/assets</code> · GET <code>/assets/:id</code>	session · registrar	inventory / register on-chain / passport
POST <code>/api/v1/assets/:id/enroll-</code> <code>tag</code>	owner/admin	enroll tag last read by a bound device (≤30 s)
PUT <code>/api/v1/assets/:id/sensor-</code> <code>policy</code>	owner/admin	versioned, audited sensor thresholds
GET <code>/api/v1/public/assets/:id</code>	public	redacted read-only passport
GET <code>/api/v1/transfers</code> · POST <code>/transfers/:id/challenge</code>	session · owner/recipient	list · reissue challenge
POST <code>/api/v1/actions/:name</code>	dev session	whitelisted lifecycle action signed by the session's dev key (<code>USE_WALLET</code> for wallets)
POST <code>/api/v1/incidents/:id/reports</code> · <code>/maintenance-evidence</code>	session / assigned technician	report hash · anchor fresh nominal maintenance evidence
GET <code>/api/v1/incidents</code>	session	incidents, reports, derived technician record

Method & path	Who	Purpose
GET/POST /api/v1/devices · /devices/:id/telemetry · POST /devices/:id/{bind, revoke, rotate -key}	session · admin	devices, telemetry, provisioning
GET/POST /api/v1/commands	session · on-chain owner/custodian	actuator commands
GET/POST /api/v1/credentials · POST /credentials/revoke	session · issuer	on-chain credentials
GET/POST /api/v1/organizations · POST /organizations/members	session · admin	orgs
GET /api/v1/evidence · /integrity/:eventId	session	evidence list · verification against chain
POST /api/v1/integrity/sandbox · /sandbox/:id/{modify, restore}	session	tamper demo on copies only
GET /api/v1/audit · /diagnostics	session	audit log · jobs, sessions, LAN URLs
GET /api/v1/public/kit	public	real-kit (ESP32-017) status: online, transport, firmware profile, per-sensor health — drives the "good to go" celebration
GET /api/v1/demo/state[? asset=PUMP-017]	session	plain-English passport for the Live Demo (owner, holder, health, locked, sensor online)
POST /api/v1/demo/{reset, handover, alarm, trySell, repair, tamper}[? asset=PUMP-017]	session, dev mode, loopback	guided story: performs the real on-chain actions as owner/technician/inspector; returns a plain-English log with tx hashes. HARDWARE_OFFLINE if the real kit isn't connected
GET/POST /api/v1/sim	session, dev mode	in-browser SIMULATED device: status, start/stop, patch sensor values (range-checked)

Actions (/actions/:name , same map used by wallets — packages/shared/src/actions.ts): activateAsset, retireAsset, setPolicy, requestTransfer, acceptTransfer, cancelTransfer, expireTransfer, completeTransfer, acknowledgeIncident, assignMaintenance, startMaintenance, submitMaintenance, assignInspector, completeInspection .

Notable error codes: UNAUTHENTICATED, FORBIDDEN, CSRF, DEV_SIGNER_DISABLED, USE_WALLET, CHAIN_UNAVAILABLE, CONTRACT_<SolidityError> (e.g. CONTRACT_AssetBlocked, CONTRACT_SelfInspection), TX_REVERTED, NOT_OPERATIONALLY_AUTHORIZED, BAD_PARAMS, NO_FRESH_EVIDENCE, ANOMALY_PRESENT, NO_TAG_SEEN ; device: BAD_ENVELOPE, UNKNOWN_DEVICE, DEVICE_REVOKED, KEY_VERSION_MISMATCH, BAD_MAC, BAD_BASE64, PAYLOAD_TOO_LARGE, UNSUPPORTED_SCHEMA, BAD_SCHEMA, SESSION_UNKNOWN, SESSION_CLOSED, SESSION_REPLAY, SEQ_REPLAY, CONFLICTING_DUPLICATE, BINDING_MISMATCH, BINDING_VERSION_STALE, RATE_LIMITED .

Database

SQLite (`data/trustmesh.db` , WAL, foreign keys ON) through Node's built-in `node:sqlite` — no native add-on.

Migrations: `apps/api/src/db.ts` (`PRAGMA user_version`). Chain-derived rows are projections; they are rebuilt from contract reads by the indexer.

Table	Contents	Key constraints
meta	fingerprint, index cursor, seeded marker	PK key
organizations, users	orgs, identities (display name, roles, dev index)	unique name / dev_index
sessions, wallet_nonces	browser sessions (CSRF token), single-use sign-in nonces	
assets	projection + metadata, sensor policy JSON, enrolled tag ref	unique chain_key
devices, device_keys, device_bindings	provisioning, secrets (current + historical for re-verification), binding history	unique chain_key; (device, binding_version)
device_sessions	authenticated sessions	unique (device, boot_id, device_nonce) → hello replay
telemetry	raw authenticated bytes + MAC + key version, readings, flags, anomalies, delayed flag, original result	unique event_id; unique (device, session, seq)
evidence	canonical bytes, hash, status queued→submitted→confirmed/failed, tx, fingerprint	PK event_id
jobs	durable oracle outbox	unique dedupe_key
chain_txs, chain_events	submitted/confirmed/reverted txs; decoded events	PK hash; PK txhash:logIndex
transfers, challenges, incidents, reports, credentials, commands, policies	workflow state / history	
audit	every operator, oracle and device action	
sandbox	integrity-lab copies (originals are never modified)	FK evidence

Records are tagged with the deployment fingerprint; queries for chain-dependent views filter on the current fingerprint. `npm run reset:demo -- --confirm` archives the DB to `data/archive/`.

Security (implemented behaviour)

- **Device auth:** per-device random 256-bit key, HMAC-SHA256 over a versioned length-delimited preimage of the exact payload bytes; MAC checked before parsing; constant-time compare; 4 KB payload cap; schema version check; key versions and revocation enforced; per-device rate limit. Symmetric — the backend can forge; documented limitation.
- **Replay:** nonce'd hello sessions (unique), unique event ids, unique (session, seq), duplicate-content comparison, delayed samples can't satisfy challenges, command ids deduped in firmware NVS, command expiry via authenticated server time.
- **Server→device:** responses MAC'd with a separate domain tag (`TMS1`); firmware ignores unauthenticated commands/challenges.
- **Browser auth:** dev identities only when `DEV_SIGNER=true` , `NODE_ENV≠production` and chain id 31337 (startup refuses otherwise). Dev keys are the public Hardhat mnemonic, never returned by the API; only whitelisted actions are signed and only with the session's own key — no general signing endpoint. Wallet sign-in checks domain (Host), chain id, expiry, single-use nonce (5 min), signature. Cookies HttpOnly + SameSite=Strict; CSRF header token on every session mutation.
- **Authorization twice:** backend checks roles/ownership for UX; contracts are the final authority for every lifecycle action.
- **Public passport:** read-only; returns org names, lifecycle/condition, confirmed event names — no secrets, raw tag ids, device data or personal addresses for non-org users.
- **Integrity:** verification reads the on-chain commitment; cached hashes are never trusted; unavailable chain → `UNVERIFIABLE` ; tampering only on sandbox copies.
- **Network:** RPC bound to 127.0.0.1. API binds 127.0.0.1 unless `API_HOST=0.0.0.0` (needed only for Wi-Fi devices; dev-login and the dev-signer action endpoint still accept loopback connections only — the Vite proxy is loopback). Plain HTTP is for a trusted development LAN only; beyond it use HTTPS (reverse proxy with TLS) and disable `DEV_SIGNER` .
- **Secrets:** `.env` , `.local/` , `data/` , `firmware/include/secrets.h` are git-ignored.

Known gaps: no TLS on the device link; symmetric device keys; single oracle; SQLite at rest unencrypted; no rate limit on browser endpoints; RFID UIDs are clonable.

Local setup

Tested: **macOS 14 (Darwin 23.4, Apple toolchain), Node 24.16.0, npm 11.13.0, Python 3.13, PlatformIO 6.1.18** (Chromium via Playwright 1.63 for UI tests). **Not tested: Windows PowerShell, Linux** — the scripts avoid shell syntax (all Node), so they are expected to work; please report issues.

Requirements: Node **22.13+ or 24** (`.nvmrc` = 24), npm, Git. Optional: Python 3 + PlatformIO (firmware), a Chromium-based browser wallet.

macOS / Linux

```
git clone <your repo> trustmesh && cd trustmesh
npm ci
npm run setup
npm run doctor
npm run dev          # → http://localhost:3000
# second terminal (no hardware):
npm run demo:simulate      # interactive SIMULATED device
npm run demo:simulate -- --scenario full  # scripted rehearsal
```

Windows (PowerShell)

```
git clone <your repo> trustmesh; cd trustmesh
npm ci
npm run setup
npm run doctor
npm run dev
npm run demo:simulate -- --scenario full
npm run bridge -- --port COM5
```

Offline: after `npm ci` (and one `npx playwright install chromium` if you run UI tests; one PlatformIO toolchain download per profile), nothing needs the internet — fonts are bundled, the chain is local, no cloud services.

What `npm run dev` does

1. Fails clearly if ports 3000/4000 are busy (`WEB_PORT` , `API_PORT` in `.env`).
2. Reuses a chain-31337 RPC already on 8545, otherwise starts a Hardhat node bound to 127.0.0.1.
3. Deploys only if the chain lacks the manifest's deployment; if it had to redeploy it **archives** the previous DB (`data/archive/`) — the local chain is ephemeral.
4. Seeds idempotently (orgs, identities, credentials, devices, 4 assets). Device secrets persist in `.local/devices.json` .
5. Starts API (watch mode) and web; prints URLs. **Ctrl+C** stops only the processes it started.

Wi-Fi ESP32

1. `.env` : `API_HOST=0.0.0.0` (and `PUBLIC_BASE_URL=http://<laptop-ip>:3000` , `WEB_LAN=true` if phones should open QR links).
2. `npm run doctor` → copy a `BACKEND_URL` candidate into `firmware/include/secrets.h` with SSID/password, device id and secret (`.local/devices.json` for ESP32-017).
3. Flash (`WIRING.md`), restart `npm run dev` , allow Node through the firewall **for private networks only**.

USB serial

Flash with `TM_WIFI_SSID ""`, close any serial monitor, then `npm run bridge -- --list` and `npm run bridge -- --port <port>`.

Reset

`npm run reset:demo -- --confirm` — archives DB, redeploys + reseeds if a chain is running (restart `npm run dev` afterwards).

Troubleshooting

Symptom	Cause / fix
port 3000 is in use	another app — stop it or set <code>WEB_PORT</code> / <code>API_PORT</code> in <code>.env</code>
port 8545 in use by something that is not a chain-31337 RPC	stop that process (e.g. another Hardhat/Anvil with a different chain id)
UI banner "chain unavailable" / <code>/ready</code> 503	chain stopped. Restart <code>npm run dev</code> (it redeploys + archives the DB because the chain is ephemeral)
<code>DB_FINGERPRINT_MISMATCH</code>	DB belongs to another deployment → <code>npm run reset:demo -- --confirm</code>
Header says "reconnecting"	API restarting; the UI reconnects automatically with backoff
ESP32 log <code>hello failed (-1)</code>	Wi-Fi not connected: check SSID/password, 2.4 GHz network, guest/hotspot client isolation
ESP32 <code>hello failed (-11/-1)</code> with Wi-Fi up	wrong <code>TM_BACKEND_URL</code> (must be laptop LAN IP, not localhost), <code>API_HOST</code> not <code>0.0.0.0</code> , or firewall. Test: <code>curl http://<ip>:4000/health</code> from a phone/another PC
<code>BAD_MAC</code> in Devices → auth	secret/key version in <code>secrets.h</code> differs from the provisioned one (rotated?)
<code>SESSION_UNKNOWN</code>	DB was reset — the firmware re-hellos automatically
<code>BINDING_VERSION_STALE</code>	device was rebound — firmware re-hellos
Bridge: <code>Access denied</code> / <code>EACCES</code>	Linux: <code>sudo usermod -aG dialout \$USER</code> and re-login; any OS: close <code>pio device monitor</code> / Arduino IDE
Bridge: port busy / lock	another program has the port open
Bridge: no ports listed	charge-only cable, missing CP210x/CH340 driver
Garbage/ malformed JSON line	baud mismatch — both sides 115200
Upload fails "Failed to connect"	hold BOOT while upload starts; lower <code>upload_speed</code>
RC522 <code>version 0x00/0xFF FAULT</code>	3.3 V supply, SPI wiring, <code>SS=GPIO21</code> , <code>RST=GPIO22</code>
Transfer verification <code>MISSING_RFID_MATCH</code>	tag not enrolled (Passport → Enroll RFID tag) or different tag
<code>SIMULATED_EVIDENCE_REJECTED_BY_REAL_POLICY</code>	expected — PUMP-017 requires REAL hardware; rehearse on SIM-PUMP-017
<code>NO_FRESH_EVIDENCE</code> on maintenance submit	the bound device must be streaming live, anomaly-free telemetry (last 30 s)
QR opens nothing on a phone	phone's <code>localhost</code> is the phone: set <code>PUBLIC_BASE_URL=http://<laptop-ip>:3000</code> and <code>WEB_LAN=true</code>
CORS errors	use the UI via <code>http://localhost:3000</code> (Vite proxy); do not point the browser at another computer's localhost
Wallet "WRONG_NETWORK"	switch to chain 31337 / RPC http://127.0.0.1:8545 (the app offers to add it)

Judge demo (5–7 min)

Self-serve alternative: the in-app **/guide** page walks anyone through 8 missions with the in-browser SIMULATED device controls (started automatically by the API in dev mode; `SIM_AUTOSTART=false` disables it).

Prep: `npm run dev`, open <http://localhost:3000>. Hardware path: ESP32-017 flashed with CORE, tag enrolled, `npm run bridge -- --port ...` (or Wi-Fi). No hardware: use the **SIMULATED rehearsal** on SIM-PUMP-017 with `npm run demo:simulate` (clearly labelled; not hardware acceptance).

t	Step	Where / who	What judges see
0:00	Labels: REAL HARDWARE device card, LOCAL EVM · chain 31337 header, PUMP-017 owner ABC Industries, custodian = owner	Overview → trust mesh, Passport	device pulses on every authenticated sample
0:45	Scan RFID / place pump on platform	Passport readings	RFID present, distance, IR, timestamps, quality flags
1:30	Owner Request transfer → CUSTODY → Technician #42	Passport (owner)	tx toast → internal tx page
2:00	Switch → Technician #42 → Transfers → Accept	Transfers	challenge issued on-chain, countdown, LED blinks blue
2:30	Hold the tag on the reader; asset in window	device	live feed: ✓ evidence accepted → custody changed; passport stamp <code>TransferCompleted</code>
3:15	Tech opens gate: Passport → Open gate	Devices → command log	created → delivered → acknowledged ("PWM sent — position not sensed")
3:45	Anomaly: lift the pump off (ASSET_REMOVED) or IR remote at flame sensor (sensor stimulus, not fire) or potentiometer (OPERATOR_TEST). Alternative: owner lowers temperature threshold in Sensor policy — say "this is a POLICY change" (audit log shows it)	device + Incidents	local red LED / pump interlock immediately; then incident opened on-chain, condition CRITICAL
4:30	Owner tries a transfer	Passport	contract rejects: <code>AssetBlocked</code>
4:45	Owner assigns Technician #42 → tech Accept & start → device nominal → Capture fresh nominal evidence & submit	Incidents	MAINTENANCE_SUBMITTED, RECOVERY_PENDING
5:30	Owner assigns Inspector #7 (try assigning the technician → <code>SelfInspection</code>) → Inspector Approve & release	Incidents	RESOLVED, NORMAL, lifecycle back; technician record +1
6:15	Integrity lab: MATCH → Create sandbox → Modify → MISMATCH (glyphs differ) → forge cached hash → still MISMATCH → Restore → MATCH	Integrity	recomputed in browser too

Recovery paths: chain died → Ctrl+C, `npm run dev` (fresh chain, archived DB, reseeded — re-run steps); device offline → UI shows STALE/UNAVAILABLE (never synthetic); wallet wrong network → toast `WRONG_NETWORK`; scripted fallback: `npm run demo:simulate -- --scenario full` (≈10 s, SIMULATED).

Extended demo (hardware, where connected)

WET/ENV/ACT profiles: rain module wet → WETNESS_DETECTED warning; DS18B20 probe; MQ135 shows `warmup` for 2 min then relative raw; LDR changes; PIR motion info events; GPS shows `nofix` indoors (never fabricated coordinates); SD `sd0k` and `/trustmesh.ndjson`; Pump ON 10 s only when on-chain NORMAL (shows "flow not verified"); Indicate → vibration + LED.

Evaluation mapping

Criterion	Implementation	Evidence
Blockchain enforces rules (≈80 %)	3 contracts: permissions, custody≠ownership, challenge-bound evidence, freshness, provenance, single-use evidence, incidents, maintenance, independent inspection, credentials	24 contract tests; e2e custody/incident; <code>CONTRACT_*</code> errors surfaced in UI
Real IoT evidence (≈20 %)	ESP32 firmware (6 profiles, 19 component types), HMAC protocol, Wi-Fi + USB bridge	all profiles compile; native vector tests; PTY bridge test; no physical test yet
Integrity	canonical evidence + keccak256 commitments; verifier reads chain; sandbox tamper lab	e2e integrity + UI test (MATCH→MISMATCH→MATCH, forged cache still mismatch)
Security	per-device keys, replay protection, CSRF, SIWE-style login, loopback-only dev signer, redacted public passport	e2e protocol/auth tests
Reliability	durable outbox with crash reconciliation, indexer with cursor, SSE resume, chain-reset detection	outbox unit tests; e2e API restart, chain outage, offline device
Usability / UX	live trust-mesh map, evidence pipeline, hash glyphs, passport stamps, role-sensitive actions, QR	screenshots in docs/screenshots
Honesty	REAL vs SIMULATED and LOCAL EVM labels everywhere; stale ≠ normal; ack ≠ physical effect	UI tests (stale device), docs

Wiring

Status: designed and compile-checked, not physically validated. A successful compile proves nothing about voltage, polarity or motion. Check every module's actual marking and datasheet before power-up.

Reference board: **ESP32-DevKitC with ESP32-WROOM-32** (classic ESP32). Not valid for ESP32-C3/S2/S3, and not for WROVER modules (GPIO16/17 are PSRAM). Constraints used ([Espressif GPIO docs](#), [ADC docs](#)):

- GPIO 6–11 → internal flash (never used). GPIO 34/35/36/39 input-only, **no internal pull-ups**.
- Strapping pins 0, 2, 5, 12, 15 avoided for external signals (GPIO0 only reads the on-board BOOT button after boot = manual stop).
- ADC2 is unusable while Wi-Fi runs → **every analog input is on ADC1** (32–39).
- GPIO is **not 5 V tolerant**: HC-SR04 ECHO and MQ135 AO go through 10 kΩ/20 kΩ dividers.

Newrro kit (ESP32-S3) — the hardware you actually have

Your Newrro controller box + sensor breakout board use fixed, labelled ports; the chip is an **ESP32-S3** (inferred from the IO45–IO48 labels — confirm with `esptool.py chip_id`). Firmware builds `newrro_min` / `newrro_full` / `newrro_full_uart`; pin map in [WIRING_TABLES.md](#) (profiles NEWRRO_MIN / NEWRRO_FULL). **Illustrated step-by-step guide with your photos:** [TRUSTMESH_Hardware_Setup_Guide.pdf](#).

Single source of truth

`firmware/pins/profiles.json` → `npm run pins` generates `firmware/include/pins_generated.h`, [WIRING_TABLES.md](#) and `wiring/*.svg`. `npm run test:pins` fails if any file is stale or a profile has: a duplicate GPIO (legitimate shared SPI bus lines excepted), output on an input-only pin, a flash or strapping pin, analog on ADC2, or a 5 V signal without a divider/shifter.

Profiles

Profile	Board	Purpose
CORE	1 ESP32	full digital lifecycle demo: RFID, HC-SR04, IR, DHT22, flame (+ servo gate, RGB)
ENV	1 ESP32	DHT22, MQ135, LDR, PIR, flame, potentiometer, RGB
WET	1 ESP32	rain, soil (intermittent power), DS18B20, HC-SR04, IR, RGB
ACT	1 ESP32	relay→pump, servo, vibration, GPS, microSD, potentiometer, RGB
FULL_A + FULL_B	2 ESP32	every component simultaneously; both bound to PUMP-017 (seeded as ESP32-017 and ESP32-017B)

Power (read before connecting motors)

- ESP32 via USB. 3.3 V rail powers RC522 (**3.3 V only**), DHT22, IR, flame, LDR, pot, DS18B20, wet sensors.
- 5 V (USB VIN) for HC-SR04, PIR, MQ135 heater (~150 mA) — check USB budget.
- **Servo: separate 5–6 V ≥2.5 A supply** (MG995/MG996R stall current ≈2.5 A), 470–1000 μF at the servo. **Pump: separate 6–12 V ≥1.5 A supply** (R385 start-up current is well above running current). Vibration motor via transistor from 5 V. **Never power motors from a GPIO or the ESP32 3V3 pin.**
- **Common ground** between ESP32 and every external supply (the relay's optocoupler isolation is only real if JD-VCC is separately supplied and the jumper removed).
- Flyback diodes: 1N5819/1N4007 across the pump, 1N4148 across the vibration motor. Relay module usually includes its coil diode — verify.

- Relay input: many 5 V opto modules are not fully OFF with a 3.3 V HIGH. Use an NPN (2N2222/BC547, 1 kΩ base) or a 3.3 V-trigger module; set `TM_RELAY_ACTIVE_LOW` to match and verify with a multimeter **before** connecting the pump.
- Low-voltage demonstration loads only. **No mains wiring.** Keep water and electronics physically separated; never run the pump dry (firmware caps a command at 30 s and never auto-starts).

CORE profile (text diagram)

```

ESP32-DevKitC                                RC522 (3.3 V!)
3V3 ----- VCC                                DHT22: VCC 3V3, DATA→GPIO4 (+10k to 3V3 if bare),
GND ----- GND                                IR: VCC 3V3, DO→GPIO35 (+100k to 3V3), GND
GPIO18 – SCK      GPIO23 – MOSI      Flame: VCC 3V3, DO→GPIO39 (+100k to GND), GND
GPIO19 – MISO      GPIO21 – SDA(SS)   HC-SR04: VCC 5V, TRIG→GPIO26,
GPIO22 – RST      (IRQ unconnected)   ECHO→10k→GPIO34→20k→GND
GPIO13 – servo signal (servo V+ = external 5–6 V, servo GND = common GND)
GPIO25/33/32 –220Ω– R/G/B cathodes; LED common anode → 3V3
GPIO0 = on-board BOOT button = manual stop (latched; clear via CLEAR_LOCAL_STOP command)

```

Full per-profile tables with supply, logic level, required circuit and firmware symbol: [WIRING_TABLES.md](#).

Safe demonstration stimuli

- Flame module: a TV-remote IR LED or phone camera IR — a **sensor stimulus, not fire**. Never use a flame, smoke or gas.
- Removal: lift the tagged pump off the platform (ASSET_REMOVED after 3 samples).
- Temperature: prefer the audited policy change (lower thresholds) over heating anything; label it as a policy change.
- Operator test: turn the potentiometer past ~85 % (labelled OPERATOR_TEST_INPUT).

Build / upload / monitor

```

python3 -m pip install --user platformio      # once
cd firmware
cp include/secrets.example.h include/secrets.h # npm run setup does this; fill device id + secret
pio run -e core                               # or env | wet | act | full_a | full_b
pio run -e core -t upload --upload-port /dev/cu.usbserial-XXXX # Windows: COM5
pio device monitor -b 115200                  # close before `npm run bridge`
pio test -e native                            # host protocol tests

```

Diagnostic mode over serial (monitor or bridge): `!diag on`, `!only tempC`, `!led r|g|b|off`, `!servo 90`, `!relay on|off`, `!vib`.

Manual hardware acceptance checklist (none executed yet)

- ☐ Board identified as WROOM-32 DevKitC; 3.3 V rail measured
- ☐ RC522 reports version 0x91/0x92 in log; tag UID shown; `Enroll RFID tag` succeeds
- ☐ HC-SR04 distance within ± 1 cm vs ruler at 10/20 cm; ECHO pin measured ≤ 3.3 V
- ☐ IR/flame DO polarity confirmed; disconnecting IR shows absent, disconnecting flame shows flame-like
- ☐ DHT22 plausible vs reference thermometer; unplugging yields null + fault
- ☐ DS18B20 –127/85 °C handling observed; probe reading vs reference
- ☐ MQ135 warm-up flag clears after 2 min; values relative only
- ☐ Rain/soil powered only during sampling (scope/LED)
- ☐ Relay OFF at boot, reset, link loss; polarity verified before pump connection

- ☐ Pump runs $\leq$ commanded seconds; flame interlock stops it immediately
- ☐ Servo end stops verified without load; startup doesn't move gate
- ☐ GPS shows `no fix` indoors; fix outdoors with HDOP
- ☐ SD mounted; `trustmesh.ndjson` grows at ≤ 1 line / 10 s
- ☐ Full custody workflow with real tag $\rightarrow$ on-chain completion

Wiring tables (generated)

Reference board: **ESP32-DevKitC (ESP32-WROOM-32, classic ESP32)** · logic 3.3 V, GPIO NOT 5 V tolerant.

ADC2 cannot be used while Wi-Fi is active: every analog input is on ADC1. GPIO12 strapping sets flash voltage — never pulled high at boot. Input-only pins have no internal pull-ups.

Profile CORE

Board: **ESP32-DevKitC (ESP32-WROOM-32, classic ESP32)**

Minimum custody demo: RFID + presence (HC-SR04, IR) + DHT22 + flame; optional servo gate + RGB status.

Capabilities: RFID, DHT22, ULTRASONIC, IR, FLAME, SERVO, RGB · Diagram: [wiring/CORE.svg](#)

Component	Signal	Dir	ESP32 GPIO	Supply	Logic level	Required circuit	Firmware symbol
RC522 RFID	SCK	out	18 (SPI_VSPI, shared)	3.3 V ONLY	3.3 V	direct	PIN_SPI_SCK
RC522 RFID	MISO	in	19 (SPI_VSPI, shared)	3.3 V	3.3 V	direct	PIN_SPI_MISO
RC522 RFID	MOSI	out	23 (SPI_VSPI, shared)	3.3 V	3.3 V	direct	PIN_SPI_MOSI
RC522 RFID	SDA/SS (chip select)	out	21	3.3 V	3.3 V	direct	PIN_RFID_SS
RC522 RFID	RST	out	22	3.3 V	3.3 V	direct; IRQ not connected	PIN_RFID_RST
DHT22	DATA	io	4	3.3 V	3.3 V	10 k Ω pull-up DATA→3.3 V if the module has none	PIN_DHT
HC-SR04	TRIG	out	26	5 V	3.3 V drive (accepted by most modules — verify)	direct	PIN_US_TRIG
HC-SR04	ECHO	in	34	5 V	5 V → 3.3 V	divider ECHO—10 k Ω —GPIO34—20 k Ω —GND ($\approx$ 3.3 V)	PIN_US_ECHO
IR obstacle module	DO (active LOW)	in	35	3.3 V	3.3 V	power module from 3.3 V; if powered from 5 V add 10 k/20 k divider; 100 k Ω pull-UP on DO so a disconnected module reads 'absent' (conservative)	PIN_IR
Flame module	DO (active LOW)	in	39	3.3 V	3.3 V	power from 3.3 V; set threshold pot with a safe IR stimulus (TV remote), not a flame; 100 k Ω pull-DOWN on DO so a disconnected module reads 'flame-like' (fail-safe, visible)	PIN_FLAME
MG995/MG996R servo	PWM	out	13	EXTERNAL 5–6 V $\geq$ 2.5 A (stall)	3.3 V PWM	servo V+ from external supply; common GND with ESP32; 470–1000 μ F bulk cap at servo; never from ESP32 5V/3V3 pin	PIN_SERVO
RGB LED (common anode)	R	out	25	anode → 3.3 V	active LOW	220 Ω series	PIN_LED_R

Component	Signal	Dir	ESP32 GPIO	Supply	Logic level	Required circuit	Firmware symbol
RGB LED (common anode)	G	out	33	anode → 3.3 V	active LOW	220 Ω series (green/blue: 100–150 Ω for brightness)	PIN_LED_G
RGB LED (common anode)	B	out	32	anode → 3.3 V	active LOW	220 Ω series	PIN_LED_B
BOOT button (on board)	manual stop	in	0	board	active LOW	on-board button; read only after boot	PIN_MANUAL_STOP

Profile ENV

Board: **ESP32-DevKitC (ESP32-WROOM-32, classic ESP32)**

Environmental evidence: DHT22, MQ135 (relative), LDR, PIR, flame, potentiometer test input, RGB.

Capabilities: DHT22, MQ135, LDR, PIR, FLAME, POT, RGB · Diagram: [wiring/ENV.svg](#)

Component	Signal	Dir	ESP32 GPIO	Supply	Logic level	Required circuit	Firmware symbol
DHT22	DATA	io	4	3.3 V	3.3 V	10 kΩ pull-up if bare sensor	PIN_DHT
MQ135	AO	in	36	5 V heater (~150 mA)	0–5 V → 0–3.3 V	divider AO—10 kΩ—GPIO36—20 kΩ—GND; 24–48 h burn-in + 2 min warm-up; raw/relative only	PIN_MQ135
LDR	divider mid-point	in	39	3.3 V	analog	3.3 V—LDR—GPIO39—10 kΩ—GND	PIN_LDR
HC-SR501 PIR	OUT	in	27	5 V (onboard 3.3 V regulator)	3.3 V output	direct; ~60 s warm-up; motion ≠ identity	PIN_PIR
Flame module	DO (active LOW)	in	35	3.3 V	3.3 V	direct; 100 kΩ pull-DOWN on DO so a disconnected module reads 'flame-like' (fail-safe, visible)	PIN_FLAME
10 kΩ potentiometer	wiper	in	34	3.3 V	analog	ends to 3.3 V/GND; OPERATOR TEST INPUT only	PIN_POT
RGB LED (common anode)	R	out	25	anode → 3.3 V	active LOW	220 Ω	PIN_LED_R
RGB LED (common anode)	G	out	33	anode → 3.3 V	active LOW	220 Ω	PIN_LED_G
RGB LED (common anode)	B	out	32	anode → 3.3 V	active LOW	220 Ω	PIN_LED_B

Profile WET

Board: **ESP32-DevKitC (ESP32-WROOM-32, classic ESP32)**

Wetness & probe temperature: rain, soil moisture (intermittently powered), DS18B20, presence, RGB.

Capabilities: RAIN, SOIL, DS18B20, ULTRASONIC, IR, RGB · Diagram: [wiring/WET.svg](#)

Component	Signal	Dir	ESP32 GPIO	Supply	Logic level	Required circuit	Firmware symbol
Rain-drop module	AO	in	36	3.3 V via PWR_WET	analog	module VCC from GPIO16 switch (≤10 mA) — or via PNP/MOSFET if the module draws more	PIN_RAIN

Component	Signal	Dir	ESP32 GPIO	Supply	Logic level	Required circuit	Firmware symbol
Soil-moisture module	AO	in	39	3.3 V via PWR_WET	analog	powered only while sampling to limit electrolysis; relative values in a test medium	PIN_SOIL
Wet-sensor power switch	VCC enable	out	16	3.3 V	active HIGH	measure module current first; use a transistor switch if >10 mA	PIN_WET_PWR
DS18B20 probe	DQ	io	4	3.3 V	3.3 V	4.7 kΩ pull-up DQ→3.3 V (not parasite power)	PIN_ONEWIRE
HC-SR04	TRIG	out	26	5 V	3.3 V drive	direct	PIN_US_TRIG
HC-SR04	ECHO	in	34	5 V	5 V → 3.3 V	10 k/20 k divider	PIN_US_ECHO
IR obstacle module	DO	in	35	3.3 V	3.3 V	direct; 100 kΩ pull-UP on DO so a disconnected module reads 'absent' (conservative)	PIN_IR
RGB LED (common anode)	R	out	25	anode → 3.3 V	active LOW	220 Ω	PIN_LED_R
RGB LED (common anode)	G	out	33	anode → 3.3 V	active LOW	220 Ω	PIN_LED_G
RGB LED (common anode)	B	out	32	anode → 3.3 V	active LOW	220 Ω	PIN_LED_B

Profile ACT

Board: **ESP32-DevKitC (ESP32-WROOM-32, classic ESP32)**

Actuation / location / storage: relay→pump, servo gate, vibration motor, GPS, SD buffer, potentiometer, RGB.

Capabilities: RELAY, PUMP, SERVO, VIBRATION, GPS, SD, POT, RGB · Diagram: [wiring/ACT.svg](#)

Component	Signal	Dir	ESP32 GPIO	Supply	Logic level	Required circuit	Firmware symbol
Relay module (5 V, optocoupler)	IN	out	27	5 V (JD-VCC)	active LOW (verify)	3.3 V HIGH may not fully turn a 5 V opto input OFF: drive IN through an NPN (2N2222/BC547, 1 kΩ base) or a 3.3 V-compatible module; verify polarity before connecting the pump	PIN_RELAY
R385 pump (via relay)	load	none	—	EXTERNAL 6–12 V ≥1.5 A	—	relay COM/NO in series with pump +; 1N5819/1N4007 flyback diode across pump; never run dry; max 30 s per command	—
MG995/MG996R servo	PWM	out	13	EXTERNAL 5–6 V ≥2.5 A	3.3 V PWM	common GND; bulk cap	PIN_SERVO
Coin vibration motor	driver gate/base	out	17	5 V via transistor	active HIGH	NPN/logic MOSFET low-side, 1 kΩ base, 1N4148 flyback, 100 nF across motor — never from GPIO	PIN_VIBRATION
NEO-6M GPS	TX → ESP RX2	in	16	3.3–5 V (module regulator)	3.3 V UART	9600 baud; ESP TX→GPS RX not required; needs sky view (indoor = no fix)	PIN_GPS_RX
microSD module (SPI)	SCK	out	18 (SPI_VSPI, shared)	per module (3.3 V or 5 V with onboard regulator + level shifter)	3.3 V	a socket/module is REQUIRED — the card cannot connect to bare GPIO	PIN_SPI_SCK

Component	Signal	Dir	ESP32 GPIO	Supply	Logic level	Required circuit	Firmware symbol
microSD module (SPI)	MISO	in	19 (SPI_VSPI, shared)	module	3.3 V	some 5 V modules do not tri-state MISO — do not share with RC522 on such modules	PIN_SPI_MISO
microSD module (SPI)	MOSI	out	23 (SPI_VSPI, shared)	module	3.3 V	direct	PIN_SPI_MOSI
microSD module (SPI)	CS	out	4	module	3.3 V	direct; 10 kΩ pull-up recommended	PIN_SD_CS
10 kΩ potentiometer	wiper	in	34	3.3 V	analog	operator test input	PIN_POT
RGB LED (common anode)	R	out	25	anode → 3.3 V	active LOW	220 Ω	PIN_LED_R
RGB LED (common anode)	G	out	33	anode → 3.3 V	active LOW	220 Ω	PIN_LED_G
RGB LED (common anode)	B	out	32	anode → 3.3 V	active LOW	220 Ω	PIN_LED_B
BOOT button (on board)	manual stop	in	0	board	active LOW	on-board	PIN_MANUAL_STOP

Profile FULL_A

Board: **ESP32-DevKitC (ESP32-WROOM-32, classic ESP32)**

FULL simultaneous design, board A (sensing). Pairs with FULL_B on a SECOND ESP32 (not in the supplied inventory). Both bind to PUMP-017.

Capabilities: RFID, DHT22, ULTRASONIC, IR, FLAME, MQ135, RAIN, SOIL, DS18B20 · Diagram: [wiring/FULL_A.svg](#)

Component	Signal	Dir	ESP32 GPIO	Supply	Logic level	Required circuit	Firmware symbol
RC522 RFID	SCK	out	18 (SPI_VSPI, shared)	3.3 V ONLY	3.3 V	direct	PIN_SPI_SCK
RC522 RFID	MISO	in	19 (SPI_VSPI, shared)	3.3 V	3.3 V	direct	PIN_SPI_MISO
RC522 RFID	MOSI	out	23 (SPI_VSPI, shared)	3.3 V	3.3 V	direct	PIN_SPI_MOSI
RC522 RFID	SDA/SS	out	21	3.3 V	3.3 V	direct	PIN_RFID_SS
RC522 RFID	RST	out	22	3.3 V	3.3 V	direct	PIN_RFID_RST
DHT22	DATA	io	4	3.3 V	3.3 V	10 kΩ pull-up if bare	PIN_DHT
HC-SR04	TRIG	out	26	5 V	3.3 V drive	direct	PIN_US_TRIG
HC-SR04	ECHO	in	34	5 V	5 V → 3.3 V	10 k/20 k divider	PIN_US_ECHO
IR obstacle module	DO	in	35	3.3 V	3.3 V	direct; 100 kΩ pull-UP on DO so a disconnected module reads 'absent' (conservative)	PIN_IR

Component	Signal	Dir	ESP32 GPIO	Supply	Logic level	Required circuit	Firmware symbol
Flame module	DO	in	39	3.3 V	3.3 V	direct; 100 kΩ pull-DOWN on DO so a disconnected module reads 'flame-like' (fail-safe, visible)	PIN_FLAME
MQ135	AO	in	36	5 V heater	0–5 V → 3.3 V	10 k/20 k divider	PIN_MQ135
Rain-drop module	AO	in	32	3.3 V via PWR_WET	analog	intermittent power	PIN_RAIN
Soil-moisture module	AO	in	33	3.3 V via PWR_WET	analog	intermittent power	PIN_SOIL
Wet-sensor power switch	VCC enable	out	25	3.3 V	active HIGH	transistor if >10 mA	PIN_WET_PWR
DS18B20 probe	DQ	io	27	3.3 V	3.3 V	4.7 kΩ pull-up	PIN_ONEWIRE

Profile FULL_B

Board: **ESP32-DevKitC (ESP32-WROOM-32, classic ESP32)**

FULL simultaneous design, board B (actuation/location/storage/indication). Second ESP32 is an ADDITIONAL part.

Capabilities: RELAY, PUMP, SERVO, VIBRATION, GPS, SD, RGB, POT, PIR, LDR · Diagram: [wiring/FULL_B.svg](#)

Component	Signal	Dir	ESP32 GPIO	Supply	Logic level	Required circuit	Firmware symbol
Relay module (5 V, optocoupler)	IN	out	27	5 V (JD-VCC)	active LOW (verify)	NPN driver recommended (see ACT)	PIN_RELAY
R385 pump (via relay)	load	none	—	EXTERNAL 6–12 V ≥1.5 A	—	flyback diode; max 30 s; never dry	—
MG995/MG996R servo	PWM	out	13	EXTERNAL 5–6 V ≥2.5 A	3.3 V PWM	common GND; bulk cap	PIN_SERVO
Coin vibration motor	driver	out	17	5 V via transistor	active HIGH	NPN + flyback	PIN_VIBRATION
NEO-6M GPS	TX → ESP RX2	in	16	3.3–5 V	3.3 V UART	9600 baud	PIN_GPS_RX
microSD module (SPI)	SCK	out	18 (SPI_VSPI, shared)	module	3.3 V	module required	PIN_SPI_SCK
microSD module (SPI)	MISO	in	19 (SPI_VSPI, shared)	module	3.3 V	direct	PIN_SPI_MISO
microSD module (SPI)	MOSI	out	23 (SPI_VSPI, shared)	module	3.3 V	direct	PIN_SPI_MOSI
microSD module (SPI)	CS	out	4	module	3.3 V	10 kΩ pull-up	PIN_SD_CS
RGB LED (common anode)	R	out	25	anode → 3.3 V	active LOW	220 Ω	PIN_LED_R
RGB LED (common anode)	G	out	33	anode → 3.3 V	active LOW	220 Ω	PIN_LED_G
RGB LED (common anode)	B	out	32	anode → 3.3 V	active LOW	220 Ω	PIN_LED_B
10 kΩ potentiometer	wiper	in	34	3.3 V	analog	operator test input	PIN_POT

Component	Signal	Dir	ESP32 GPIO	Supply	Logic level	Required circuit	Firmware symbol
HC-SR501 PIR	OUT	in	35	5 V	3.3 V	direct	PIN_PIR
LDR	divider	in	39	3.3 V	analog	LDR + 10 kΩ divider	PIN_LDR
BOOT button (on board)	manual stop	in	0	board	active LOW	on-board	PIN_MANUAL_STOP

Profile NEWRRO_MIN

Board: **Newrro controller with ESP32-S3 (inferred from IO45–IO48 labels — verify with the chip-id step)**

Newrro kit, minimum: RC522 + DS18B20 + onboard LDR + onboard RGB + IR presence. PUMP-017 policy must be set to RFID_MATCH + PRESENCE (no flame sensor).

Capabilities: RFID, DS18B20, LDR, RGB, IR · Diagram: [wiring/NEWRRO_MIN.svg](#)

Component	Signal	Dir	ESP32 GPIO	Supply	Logic level	Required circuit	Firmware symbol
RC522 RFID (Newrro RFID port)	3V3 supply	none	—	3.3 V (port pin 1)	—	plug RC522 into the 8-pin RFID port; RC522 must NOT get 5 V	—
RC522 RFID (Newrro RFID port)	RST	out	15	3.3 V	3.3 V	board trace (port label IO15)	PIN_RFID_RST
RC522 RFID (Newrro RFID port)	MISO	in	16 (SPI_NEWRRO, shared)	3.3 V	3.3 V	board trace (IO16) — function inferred from RC522 pin order, verify	PIN_SPI_MISO
RC522 RFID (Newrro RFID port)	MOSI	out	17 (SPI_NEWRRO, shared)	3.3 V	3.3 V	board trace (IO17) — inferred, verify	PIN_SPI_MOSI
RC522 RFID (Newrro RFID port)	SCK	out	18 (SPI_NEWRRO, shared)	3.3 V	3.3 V	board trace (IO18) — inferred, verify	PIN_SPI_SCK
RC522 RFID (Newrro RFID port)	SDA/SS	out	3	3.3 V	3.3 V	board trace (IO3); GPIO3 is a strapping pin chosen by the board maker — output only after boot	PIN_RFID_SS
DS18B20 probe (TEMP port)	DQ	io	1	5 V port pin (probe works 3–5.5 V)	3.3 V (board pull-up — verify)	already plugged into the TEMP port in your photo; board has the pull-up resistor	PIN_ONEWIRE
LDR (onboard)	divider	in	4	3.3 V	analog	soldered on the breakout board with its resistor	PIN_LDR
RGB LED (onboard, common cathode)	R	out	5	board	active HIGH	onboard; colour order inferred — check with lled r	PIN_LED_R
RGB LED (onboard, common cathode)	G	out	6	board	active HIGH	onboard	PIN_LED_G

Component	Signal	Dir	ESP32 GPIO	Supply	Logic level	Required circuit	Firmware symbol
RGB LED (onboard, common cathode)	B	out	7	board	active HIGH	onboard	PIN_LED_B
IR obstacle module	OUT (active LOW)	in	12	5 V from the port	MEASURE OUT first (see guide step 4)	3-pin port 5V/IO12/GND; internal pull-up enabled; if OUT measures above 3.6 V use a 10k/20k divider	PIN_IR

Profile NEWRRO_FULL

Board: **Newrro controller with ESP32-S3 (inferred from IO45–IO48 labels — verify with the chip-id step)**

Newrro kit + flame module (IO13) + DHT22 (IO14): satisfies the default PUMP-017 custody policy (RFID_MATCH + PRESENCE + NO_FLAME).

Capabilities: RFID, DS18B20, LDR, RGB, IR, FLAME, DHT22 · Diagram: [wiring/NEWRRO_FULL.svg](#)

Component	Signal	Dir	ESP32 GPIO	Supply	Logic level	Required circuit	Firmware symbol
RC522 RFID (Newrro RFID port)	3V3 supply	none	—	3.3 V (port pin 1)	—	plug RC522 into the 8-pin RFID port; RC522 must NOT get 5 V	—
RC522 RFID (Newrro RFID port)	RST	out	15	3.3 V	3.3 V	board trace (port label IO15)	PIN_RFID_RST
RC522 RFID (Newrro RFID port)	MISO	in	16 (SPI_NEWRRO, shared)	3.3 V	3.3 V	board trace (IO16) — function inferred from RC522 pin order, verify	PIN_SPI_MISO
RC522 RFID (Newrro RFID port)	MOSI	out	17 (SPI_NEWRRO, shared)	3.3 V	3.3 V	board trace (IO17) — inferred, verify	PIN_SPI_MOSI
RC522 RFID (Newrro RFID port)	SCK	out	18 (SPI_NEWRRO, shared)	3.3 V	3.3 V	board trace (IO18) — inferred, verify	PIN_SPI_SCK
RC522 RFID (Newrro RFID port)	SDA/SS	out	3	3.3 V	3.3 V	board trace (IO3); GPIO3 is a strapping pin chosen by the board maker — output only after boot	PIN_RFID_SS
DS18B20 probe (TEMP port)	DQ	io	1	5 V port pin (probe works 3–5.5 V)	3.3 V (board pull-up — verify)	already plugged into the TEMP port in your photo; board has the pull-up resistor	PIN_ONE_WIRE
LDR (onboard)	divider	in	4	3.3 V	analog	soldered on the breakout board with its resistor	PIN_LDR
RGB LED (onboard, common cathode)	R	out	5	board	active HIGH	onboard; colour order inferred — check with !led r	PIN_LED_R

Component	Signal	Dir	ESP32 GPIO	Supply	Logic level	Required circuit	Firmware symbol
RGB LED (onboard, common cathode)	G	out	6	board	active HIGH	onboard	PIN_LED_G
RGB LED (onboard, common cathode)	B	out	7	board	active HIGH	onboard	PIN_LED_B
IR obstacle module	OUT (active LOW)	in	12	5 V from the port	MEASURE OUT first (see guide step 4)	3-pin port 5V/IO12/GND; internal pull-up enabled; if OUT measures above 3.6 V use a 10k/20k divider	PIN_IR
Flame module	DO (active LOW)	in	13	5 V from the port	MEASURE DO first	3-pin port 5V/IO13/GND; internal pull-down = fail-safe; if DO measures above 3.6 V use a 10k/20k divider	PIN_FLAME
DHT22	DATA	io	14	5 V from the port	open-drain (safe with 3.3 V pull-up only)	3-pin port 5V/IO14/GND; module pull-up to 5 V would give 5 V on DATA — prefer a bare sensor with 10 kΩ to 3V3	PIN_DHT

Hardware BOM

Supplied inventory (from the master prompt's table; `sensors.xlsx` and the product brief were **not present** on this machine). Quantities are stock, not per-model requirements. No prices are given.

Provided component	Additional requirement	Qty per model	Electrical purpose	Required / optional	Validation status
— (not supplied)	ESP32-DevKitC (WROOM-32) reference board	1 (CORE) · 2 (FULL)	controller, Wi-Fi, 3.3 V logic	required	firmware compiles; not hardware-tested
—	USB data cable (not charge-only)	1 per board	upload + serial bridge	required	—
—	Breadboard, jumper wires	1 set	prototyping	required	—
RC522 RFID reader	13.56 MHz MIFARE tags/cards (not supplied)	1 reader, ≥ 2 tags	asset tag read	required (CORE)	untested
HC-SR04	10 k Ω + 20 k Ω resistors (ECHO divider)	1	presence distance	required (CORE)	untested
IR obstacle module	100 k Ω pull-up on DO	1	near-field presence	required (CORE)	untested
Flame module	100 k Ω pull-down on DO; safe IR stimulus (TV remote)	1	flame-like optical input	required (CORE)	untested
DHT22	10 k Ω pull-up if bare sensor	1	ambient T/RH	required (CORE)	untested
MG995 servo (sheet link says MG996R — identify actual part)	external 5–6 V ≥ 2.5 A supply , 470–1000 μ F capacitor	1	model gate	optional (CORE/ACT)	untested; end stops unverified
RGB LED, common anode	3 $\times$ 220 Ω (or 100–150 Ω G/B)	1	status indication	optional	untested
MQ135	10 k Ω + 20 k Ω divider; 5 V heater supply	1	relative air quality	optional (ENV)	untested; uncalibrated
LDR	10 k Ω divider resistor	1	light/cover change	optional (ENV)	untested
HC-SR501 PIR	—	1	motion near asset	optional	untested
10 k Ω potentiometer	—	1	operator test input	optional	untested
Rain-drop module	transistor switch if > 10 mA	1	wetness	optional (WET)	untested
Soil-moisture module	shares wet-power switch	1	relative moisture	optional (WET)	untested
DS18B20 probe	4.7 k Ω pull-up	1	probe temperature	optional (WET)	untested
1-ch 5 V relay (optocoupler)	NPN (2N2222/BC547) + 1 kΩ input driver recommended; verify polarity	1	pump switching (low voltage only)	optional (ACT)	untested
R385 pump	external 6–12 V ≥ 1.5 A supply , 1N5819/1N4007 flyback diode, tubing, water container	1	demo asset / actuation	optional (wet demo)	untested
Coin vibration motor	NPN/logic MOSFET, 1 k Ω , 1N4148 flyback, 100 nF	1	haptic incident	optional (ACT)	untested
NEO-6M GPS	antenna with sky view	1	location evidence	optional (ACT)	untested (no fix indoors expected)
SanDisk 8 GB card	microSD SPI socket module (card cannot connect to bare GPIO)	1	offline evidence log	optional	untested

Provided component	Additional requirement	Qty per model	Electrical purpose	Required / optional	Validation status
—	Common GND between ESP32 and all external supplies	—	reference	required	—

FULL simultaneous design = FULL_A + FULL_B on **two** ESP32 boards (second board is an additional part). No I/O expander is required for the documented split.

Hardware coverage — all 19 supplied component types

"Automated test" = what software checks cover the path. **No row has been physically tested** — no ESP32 or modules were connected during this build. Compilation of every profile was verified (`npm run firmware:build` , PlatformIO `espressif32@6.9.0`).

#	Component	Firmware driver	Profile(s)	Pins / interface (reference board)	Backend field	UI	Automated test	Physical test	Missing dependency
1	SanDisk 8 GB card	SD (Arduino core), rate-limited NDJSON log	ACT, FULL_B	VSPI 18/19/23, CS 4	sd0k	device readings	compile	not done	microSD socket module
2	1-ch relay	relayWrite + interlocks, max 30 s, comm-loss off, safe boot state	ACT, FULL_B	GPIO27	pumpCmd , local.pump0n	Pump ON/OFF commands, command log	compile; e2e pump command rules	not done	NPN driver; polarity verification
3	MQ135	analogRead + 120 s warm-up quality	ENV, FULL_A	GPIO36 (divider)	gasRaw (relative)	reading tile "not ppm / not AQI"	compile; policy unit test (warm-up)	not done	divider; burn-in
4	Flame module	debounced digital, local interlock	CORE, ENV, FULL_A	GPIO39 / 35	flame , NO_FLAME flag, FLAME_LIKE_OPTICAL	incident workflow	policy unit test; e2e incident (simulated)	not done	pull-down; safe IR stimulus
5	HC-SR04	bounded pulseIn (25 ms), median of 5	CORE, WET, FULL_A	TRIG 26, ECHO 34 (divider)	distanceCm , PRESENCE	readings + sparkline	policy unit test; e2e custody	not done	10k/20k divider
6	IR module	debounced digital	CORE, WET, FULL_A	GPIO35	irPresent , PRESENCE	readings	policy unit test; e2e custody	not done	pull-up
7	DHT22	Adafruit DHT, NaN → null	CORE, ENV, FULL_A	GPIO4	tempC , humidityPct , TEMP_OK	readings	policy hysteresis tests	not done	pull-up if bare
8	MG995/MG996R servo	ESP32Servo, attach-on-command, auto-detach	CORE, ACT, FULL_B	GPIO13 (LEDC)	servoDeg (commanded)	Open/Close gate, ack	compile; e2e gate command (sim)	not done	external 5–6 V supply; identify model/end stops
9	R385 pump	via relay	ACT, FULL_B	relay NO/COM	pumpCmd	"pump command ON — flow not verified"	as relay	not done	6–12 V supply, flyback diode, water loop
10	RC522	MFRC522 lib, version-register self-test, wake-up for presence	CORE, FULL_A	VSPI + SS 21, RST 22	rfidPresent , rfidTag (salted hash)	enroll tag, readings	policy unit test; e2e custody	not done	MIFARE tags

#	Component	Firmware driver	Profile(s)	Pins / interface (reference board)	Backend field	UI	Automated test	Physical test	Missing dependency
11	Rain-drop	intermittent power, AO	WET, FULL_A	GPIO36 / 32, power 16 / 25	rainRaw , NO_WET , WETNESS_DETECTED	readings	policy evaluation (e2e sim)	not done	switch transistor if >10 mA
12	Soil moisture	intermittent power, AO	WET, FULL_A	GPIO39 / 33	soilRaw (relative)	readings	compile	not done	test medium
13	DS18B20	DallasTemperature async; -127/85 °C rejected; local interlock ≥60 °C	WET, FULL_A	GPIO4 / 27	probeTemp C	readings	policy hysteresis tests	not done	4.7 kΩ pull-up
14	10 kΩ potentiometer	anaLogRead	ENV, ACT, FULL_B	GPIO34	potRaw → OPERATOR_TEST_INPUT	labelled operator test	policy unit test	not done	—
15	RGB LED (CA)	active-LOW status: red interlock / blue challenge / green linked / amber no link	CORE, ENV, WET, ACT, FULL_B	25/33/32	— (local only)	INDICATE command	compile	not done	3x resistors
16	HC-SR501 PIR	digital + 60 s warm-up	ENV, FULL_B	GPIO27 / 35	motion → info event	readings	policy (info, not identity)	not done	—
17	NEO-6M GPS	TinyGPSPlus on UART2; fix only if valid & <5 s old	ACT, FULL_B	RX2 = GPIO16	gpsFix , lat , lon , hdop (null + nofix)	readings show "nofix"	simulator path (nofix)	not done	sky view
18	LDR	divider anaLogRead	ENV, FULL_B	GPIO39	ldrRaw	readings	compile	not done	10 kΩ
19	Vibration motor	pulse on interlock / INDICATE	ACT, FULL_B	GPIO17 via transistor	— (local)	INDICATE	compile	not done	NPN/MOSFET + diode

Pin maps, per-connection circuits and diagrams: [WIRING.md](#), [WIRING_TABLES.md](#) (generated), [wiring/](#).

Deployment

Supported: local single-laptop operation via `npm run dev` (Hardhat chain 31337, SQLite, loopback RPC). This is a development chain, not a decentralized production network.

Production build check: `npm run build` (contracts compile + ABI export, API typecheck + esbuild bundle `apps/api/dist/main.mjs`, web bundle `apps/web/dist`). `NODE_ENV=production` refuses to start with `DEV_SIGNER=true`.

External EVM networks (not verified in this build): `hardhat.config.cjs` can be given another network and `scripts/deploy.cjs` writes the same manifest; the API supports `RPC_URL` and `ORACLE_PRIVATE_KEY` (holding `ORACLE_ROLE`), but registrar/admin operations (asset registration, device provisioning, credentials) currently sign with the local dev admin key only — a production registrar signer is **not implemented**. TLS in front of the API and wallets for all users would also be required. No MST network integration existed in the repository and none was invented — addresses, chain ids and explorer URLs for any external network must come from that network's official documentation. Local transactions are always labelled LOCAL EVM and linked to the internal explorer.

Acceptance matrix

Status: **T** automated test passed · **I** implemented (manually exercised or compile-only) · **H** hardware-tested · **X** not done / blocked. Evidence refers to TEST_REPORT.md suites.

\$	Requirement	Status	Evidence / location
1	Asset → device → evidence → hash → chain → decision → response	T	e2e custody + incident; scripts/scenarios.ts
1	Brief (Pasted text(6).txt) & sensors.xlsx read	X	files not present; prompt table used
2	Plan / acceptance / status docs	I	this file, IMPLEMENTATION_*
3	Root commands ci/setup/doctor/dev/demo:simulate/bridge/test:all/build/reset:demo	T	clean-copy smoke; each command run
3	Ports 3000/4000/8545 configurable, conflict detection, Ctrl+C cleanup	T	dev.mjs; manual verification
3	Dev signer local-only, no key exposure, no general signing	T	chain.ts devSignerEnabled , loopback-only, whitelist; e2e
3	Browser wallet sign-in, wrong network, wallet tx	I / T(partial)	wrong network e2e; real wallet tx not run
3	Offline runtime after install	I	bundled fonts; no network calls
3	macOS / Windows / Linux	T / X / X	macOS only
4	BOM with supplied vs additional parts	I	docs/HARDWARE_BOM.md
5	CORE, ENV, WET, ACT profiles; FULL two-board design	I C	firmware:build 6/6
5	19-component matrix	I	docs/HARDWARE_COVERAGE.md
5/6	Physical demonstration & wiring validated	X	no hardware available
6	Pin/power matrix, dividers, drivers, safety, diagrams generated from config, conflict checker	I T	profiles.json, gen_pins.mjs, test:pins
7	Nonblocking sensing, null on fault, quality flags, warm-up, debounce/median, safe outputs, bounded buffer, stable ids, diag mode, Wi-Fi backoff + serial fallback	I C	firmware/src
8	HMAC envelope over exact bytes, vectors shared with firmware	T	test:shared, firmware:test
8	Sessions, replay, duplicates, delayed data, binding checks, key rotation/revocation	T	e2e protocol tests
8	Canonical evidence + keccak256; provenance from registry	T	shared tests; contract tests
9	Three contracts, orthogonal states, all listed rules, pause semantics, manifest	T	24 contract tests
9	MST integration	n/a	none existed; none invented
10	Exact custody story incl. challenge, device evidence, on-chain completion, gate command ack	T	e2e (simulated device) · H X
10	Ownership transfer with acceptance	T	contract test
11	Versioned sensor policy, hysteresis, dedup incidents, acknowledge ≠ resolve, maintenance → independent inspection, credentials, disclosed reputation	T	api policy tests, contract + e2e
12	Local protective response before chain; authenticated expiring commands; ack ≠ effect; pump limits	I T(backend)	firmware main.cpp; e2e command tests
13	Durable outbox, nonce serialization, crash reconciliation, indexer cursor, chain-reset detection, restart resume	T	outbox tests; e2e restart/outage
14	Validated versioned API, OpenAPI, CSRF, wallet nonce/domain/chain/expiry, public redaction	T	docs/openapi.json; e2e auth
15	All screens functional, REAL/SIMULATED + LOCAL EVM labels, pipeline, internal tx page, QR with LAN base URL	T(key flows) / I	e2e UI; screenshots

§	Requirement	Status	Evidence / location
16	Integrity: chain read, recompute, sandbox tamper, forged cache, UNVERIFIABLE when chain down	T	e2e
17	Wi-Fi & USB paths, doctor, troubleshooting	I / T(USB via PTY)	bridge test
18	Tests & report	T	TEST_REPORT.md
20	Deliverable docs	I	docs/
21	Judge demo script + simulator rehearsal	I T(sim)	docs/DEMO.md

Test report

Run: **2026-09-28T21:49Z** on macOS 14 (Darwin 23.4), Node 24.16.0, npm 11.13.0, Hardhat 2.29.1, solc 0.8.28, OpenZeppelin 5.6.1, viem 2.56.9, Fastify 5.12.5, React 19.3.0, Vite 7.3.6, Tailwind 4.3.3, Playwright 1.63.0 (Chromium), PlatformIO 6.1.18 / espressif32 6.9.0 (Arduino-ESP32 2.0.17).

Command	Result
<code>npm run test:pins</code> (generator freshness + conflict checker unit tests)	✓ all 6 profiles valid, 19/19 capabilities covered; 5/5 checker tests pass
<code>npm run test:shared</code> (HMAC + canonical hash vectors, tamper/wrong key/domain/size)	✓ 4/4
<code>npm run test:contracts</code> (Hardhat)	✓ 24/24
<code>npm run test:api</code> (policy engine, outbox crash recovery)	✓ 10/10
<code>npm run test:e2e</code> (isolated chain 8546 + API 4100 + UI 3100, Chromium)	✓ 24/24 (≈38 s)
<code>npm run firmware:test</code> (<code>pio test -e native</code> , same C++ as device vs shared vectors)	✓ 3/3
<code>npm run firmware:build</code> (core, env, wet, act, full_a, full_b)	✓ 6/6 compile; flash 942–994 KB of 1310 KB (72–76 %), RAM ≈15 %
<code>npm run test:bridge</code> (real PTY, Python device ↔ TS bridge ↔ API)	✓ 5/5
<code>npm run build</code> (contracts, API typecheck + bundle, web bundle)	✓ (web JS 574 KB / 173 KB gzip — chunk-size warning only)
<code>npm run demo:simulate -- --scenario full</code> against <code>npm run dev</code>	✓ custody → incident → blocked transfer → maintenance → self-inspection rejected → inspection → NORMAL → integrity MATCH/MISMATCH/MISMATCH(forged cache)/MATCH
Clean-copy smoke (fresh dir, <code>npm ci</code> from lockfile → <code>setup</code> → <code>doctor</code> → <code>dev</code> → full simulation → Ctrl+C frees 3000/4000/8545)	✓
Restart behaviour	✓ <code>npm run dev</code> after Ctrl+C detects the fresh chain, archives the old DB, redeploys, reseeds; API restart keeps data and SSE reconnects (e2e)

What the suites cover

- **Contracts**: registration/roles, provenance from registry, custody vs ownership, recipient-only acceptance, one active transfer, missing/wrong-challenge/stale/pre-challenge/flag-deficient/simulated evidence, revoked & rebound devices, double completion & evidence replay, expiry/cancel, critical-after-acceptance race, incident open/update, NORMAL cannot clear, acknowledge ≠ resolve, maintenance → independent inspection → recovery, rejected inspection, credential revoke/expiry, simulated maintenance evidence, pause semantics, versioned policy.
- **e2e**: tampered MAC, wrong key/key version/unknown device/malformed envelope, size limit, unknown schema, identical retry, conflicting duplicate, seq replay, hello replay, delayed old-session data, binding mismatch, CSRF, public redaction, unauthenticated access, wallet nonce replay, custody via device evidence, custodian-only gate command + ack, full incident workflow, pump release rules, integrity lab, real-policy rejection of simulated evidence + revoked device, credential revocation, outbox settled, UI login/live, UI custody via passport & transfers pages, UI integrity lab, wrong wallet network (injected provider), SSE reconnect after API restart, offline device → STALE/UNAVAILABLE, chain outage → 503 / UI banner / UNVERIFIABLE / actions refused, no page errors.

Defects found and fixed during testing

Stale viem block-number cache broke read-your-writes; Hardhat timestamp drift vs wall clock; evidence ordering compared equal same-second timestamps (now strict block ordering in the contract); native EventSource giving up after proxy 502 (manual reconnect); live-status subscription race; API shutdown held open by SSE streams; simulator reusing event ids across sessions; parameter validation order for PUMP_ON; bridge JSON detection; Tailwind v4 component classes; transactions pre-simulated against a stale (idle) latest block → now simulated against the pending block; SSE buffered by Cloudflare quick tunnels → UI falls back to 3 s auto-refresh (verified over the public tunnel).

Not tested / limitations

- **No physical hardware tested.** No ESP32, sensor, actuator, RFID tag or wiring was available. Every hardware item in `HARDWARE_COVERAGE.md` is "not done"; compile success does not validate wiring, voltage, polarity or motion.
- Windows and Linux not executed (scripts are Node-only by design). GitHub Actions workflow written but not run.
- Browser-wallet transaction signing was exercised only up to network validation (injected mock provider); a real MetaMask transaction was not executed.
- Wi-Fi transport exercised only by code review/compile; the serial path was tested with a PTY, not a real UART.

Implementation status — 2026-09-29

All milestones implemented; automated suites green (see TEST_REPORT.md). Legend: **I** implemented · **C** compiled · **T** automated tests pass · **H** hardware-tested · **B** blocked.

Area	State
Contracts	I C T
Backend device protocol, policy, evidence, outbox, indexer, SSE, auth	I T
Web UI (all listed screens)	I T (Chromium e2e for key flows)
Simulator + scenarios	I T
Serial bridge	I T (PTY, not a real UART)
Firmware, 6 profiles	I C T (host protocol tests) — H: not done (no hardware)
Wi-Fi transport	I C — not exercised end-to-end
Browser wallet	I — sign-in + wrong-network tested with a mock provider; real wallet tx not executed
Windows / Linux	not executed
CI workflow	written, not run

Next actions (need physical access): flash CORE on a WROOM-32 DevKitC, walk the checklist in docs/WIRING.md, enroll a real tag, run the DEMO.md script with REAL HARDWARE, then extend to ENV/WET/ACT and the two-board FULL design. Identify the actual servo model (MG995 vs MG996R) and relay input polarity first.

Resume commands: `npm ci && npm run setup && npm run test:all && npm run dev`.

Endpoint reference

Generated from `docs/openapi.json` (OpenAPI 3.0.3, 59 operations). Live interactive version: <http://localhost:4000/docs>.

Health

GET `/health`

Liveness check. Always 200 while the API process runs.

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

GET `/ready`

Readiness: database, chain RPC, manifest and deployment fingerprint. 503 when the chain is unavailable or mismatched.

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

Authentication

GET `/api/v1/auth/personas`

Lists the local development identities (only when the dev signer is enabled).

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

POST `/api/v1/auth/dev-login`

Signs in as a local development identity (public Hardhat test keys; chain 31337 only; loopback only). Returns the CSRF token.

REQUEST BODY (JSON SCHEMA)

```
{
  "type": "object",
  "required": [
    "persona"
  ],
  "properties": {
    "persona": {
      "type": "string"
    }
  }
}
```

RESPONSES

200 · errors return `{"error": CODE, "message": text}`

POST /api/v1/auth/nonce

Creates a single-use sign-in message (domain, chain id, nonce, 5-minute expiry) for a wallet address.

REQUEST BODY (JSON SCHEMA)

```
{
  "type": "object",
  "required": [
    "address"
  ],
  "properties": {
    "address": {
      "type": "string",
      "pattern": "^0x[0-9a-fA-F]{40}$"
    }
  }
}
```

RESPONSES

200 · errors return {"error": CODE, "message": text}

POST /api/v1/auth/wallet

Verifies a signed sign-in message (domain, chain, expiry, unused nonce, signature) and starts a wallet session.

REQUEST BODY (JSON SCHEMA)

```
{
  "type": "object",
  "required": [
    "message",
    "signature"
  ],
  "properties": {
    "message": {
      "type": "string",
      "maxLength": 2000
    },
    "signature": {
      "type": "string"
    }
  }
}
```

RESPONSES

200 · errors return {"error": CODE, "message": text}

GET /api/v1/auth/me

Current session user (name, address, roles, CSRF token) and whether the dev signer is enabled.

RESPONSES

200 · errors return {"error": CODE, "message": text}

POST /api/v1/auth/logout

Ends the browser session.

RESPONSES

200 · errors return {"error": CODE, "message": text}

Public (no sign-in)

GET /api/v1/public/assets/{id}

Public read-only passport (what the QR code opens): no secrets, raw tag ids or private data.

Parameter	In	Required	Type
id	path	yes	string

RESPONSES

200 · errors return {"error": CODE, "message": text}

GET /api/v1/public/kit

Public real-kit status (ESP32-017): online, transport, firmware profile and per-sensor health.

RESPONSES

200 · errors return {"error": CODE, "message": text}

Device protocol (HMAC envelopes)

POST /api/v1/device/hello

Device → server (HMAC envelope with a random nonce): opens a session; the MAC'd response carries the session id, server time, asset binding and poll interval.

REQUEST BODY (JSON SCHEMA)

```
{
  "type": "object",
  "required": [
    "deviceId",
    "keyVersion",
    "payloadB64",
    "macHex"
  ],
  "additionalProperties": false,
  "properties": {
    "deviceId": {
      "type": "string",
      "maxLength": 64
    },
    "keyVersion": {
      "type": "integer",
      "minimum": 1
    },
    "payloadB64": {
      "type": "string",
      "maxLength": 6000
    },
    "macHex": {
      "type": "string",
      "maxLength": 64
    }
  }
}
```

RESPONSES

200 · errors return {"error": CODE, "message": text}

POST /api/v1/device/telemetry

Device → server (HMAC envelope): one signed sensor sample. Verified, deduplicated, evaluated against policy; may create evidence and chain transactions.

REQUEST BODY (JSON SCHEMA)

```
{
  "type": "object",
  "required": [
    "deviceId",
    "keyVersion",
    "payloadB64",
    "macHex"
  ],
  "additionalProperties": false,
  "properties": {
    "deviceId": {
      "type": "string",
      "maxLength": 64
    },
    "keyVersion": {
      "type": "integer",
      "minimum": 1
    },
    "payloadB64": {
      "type": "string",
      "maxLength": 6000
    },
    "macHex": {
      "type": "string",
      "maxLength": 64
    }
  }
}
```

RESPONSES

200 · errors return {"error": CODE, "message": text}

POST /api/v1/device/poll

Device → server (HMAC envelope): fetches pending signed commands and the active custody challenge.

REQUEST BODY (JSON SCHEMA)

```
{
  "type": "object",
  "required": [
    "deviceId",
    "keyVersion",
    "payloadB64",
    "macHex"
  ],
  "additionalProperties": false,
  "properties": {
    "deviceId": {
      "type": "string",
      "maxLength": 64
    },
    "keyVersion": {
      "type": "integer",
      "minimum": 1
    },
    "payloadB64": {
      "type": "string",
      "maxLength": 6000
    },
    "macHex": {
      "type": "string",
      "maxLength": 64
    }
  }
}
```

RESPONSES

200 · errors return {"error": CODE, "message": text}

POST /api/v1/device/ack

Device → server (HMAC envelope): acknowledges a command as applied, rejected or uncertain.

REQUEST BODY (JSON SCHEMA)

```
{
  "type": "object",
  "required": [
    "deviceId",
    "keyVersion",
    "payloadB64",
    "macHex"
  ],
  "additionalProperties": false,
  "properties": {
    "deviceId": {
      "type": "string",
      "maxLength": 64
    },
    "keyVersion": {
      "type": "integer",
      "minimum": 1
    },
    "payloadB64": {
      "type": "string",
      "maxLength": 6000
    },
    "macHex": {
      "type": "string",
      "maxLength": 64
    }
  }
}
```

RESPONSES

200 · errors return {"error": CODE, "message": text}

Live demo & simulator

GET /api/v1/demo/state

Plain-English Live Demo passport (owner, holder, health, locked, sensor online). Add ?asset=PUMP-017 for the real kit.

RESPONSES

200 · errors return {"error": CODE, "message": text}

POST /api/v1/demo/{step}

Runs one Live Demo story step — reset, handover, alarm, trySell, repair, tamper — performing the real on-chain actions as owner/technician/inspector; returns a plain-English log with transaction hashes. ?asset=PUMP-017 uses the real kit (HARDWARE_OFFLINE if it isn't connected). Dev mode, loopback only.

Parameter	In	Required	Type
step	path	yes	string

RESPONSES

200 · errors return {"error": CODE, "message": text}

GET /api/v1/sim

Status of the in-browser SIMULATED device (dev mode).

RESPONSES

200 · errors return {"error": CODE, "message": text}

POST /api/v1/sim

Starts/stops the SIMULATED device or patches its sensor values (range-checked, audited).

RESPONSES

200 · errors return {"error": CODE, "message": text}

Assets

GET /api/v1/assets

Inventory: every asset with owner, custodian, location, lifecycle, condition and policy, plus an address→name map.

RESPONSES

200 · errors return {"error": CODE, "message": text}

POST /api/v1/assets

Registers a new asset on-chain (registrar/admin). The owner must then activate it with their own transaction.

REQUEST BODY (JSON SCHEMA)

```
{
  "type": "object",
  "required": [
    "id",
    "name",
    "location",
    "owner",
    "requireReal"
  ],
  "properties": {
    "id": {
      "type": "string"
    },
    "name": {
      "type": "string",
      "minLength": 2,
      "maxLength": 80
    },
    "model": {
      "type": "string",
      "maxLength": 80
    },
    "description": {
      "type": "string",
      "maxLength": 500
    },
    "location": {
      "type": "string",
      "maxLength": 80
    },
    "owner": {
      "type": "string",
      "pattern": "^0x[0-9a-fA-F]{40}$"
    },
    "custodian": {
      "type": "string",
      "pattern": "^0x[0-9a-fA-F]{40}$"
    },
    "requireReal": {
      "type": "boolean"
    },
    "requiredFlags": {
      "type": "integer",
      "minimum": 0
    },
    "maxEvidenceAge": {
      "type": "integer",
      "minimum": 10,
      "maximum": 86400
    }
  }
}
```

RESPONSES

200 · errors return {"error": CODE, "message": text}

GET /api/v1/assets/{id}

Full passport: asset, bound devices with latest readings, transfers, incidents, evidence, bindings, policies, reports and confirmed chain events.

Parameter	In	Required	Type
id	path	yes	string

RESPONSES

200 · errors return {"error": CODE, "message": text}

POST /api/v1/assets/{id}/enroll-tag

Enrolls the RFID tag most recently read (≤ 30 s) by a bound device as the asset's identity tag. Owner or admin.

Parameter	In	Required	Type
id	path	yes	string

RESPONSES

200 · errors return {"error": CODE, "message": text}

PUT /api/v1/assets/{id}/sensor-policy

Updates the versioned, audited sensor-interpretation thresholds (a policy change, not a sensor change). Owner or admin.

Parameter	In	Required	Type
id	path	yes	string

RESPONSES

200 · errors return {"error": CODE, "message": text}

Transfers & lifecycle actions

GET /api/v1/transfers

Transfers with their challenges (status, expiry, bindings, last verification result).

RESPONSES

200 · errors return {"error": CODE, "message": text}

POST /api/v1/transfers/{id}/challenge

Owner or recipient: issues a fresh custody challenge (invalidates the previous one).

Parameter	In	Required	Type
id	path	yes	string

RESPONSES

200 · errors return {"error": CODE, "message": text}

POST /api/v1/actions/{name}

Performs a whitelisted lifecycle action (e.g. requestTransfer, acceptTransfer, assignMaintenance, completeInspection) as the signed-in development identity; returns the confirmed transaction hash. Wallet sessions get USE_WALLET and sign in the browser instead. Loopback only.

Parameter	In	Required	Type
name	path	yes	string

RESPONSES

200 · errors return {"error": CODE, "message": text}

Incidents, maintenance, inspection

POST /api/v1/incidents/{id}/reports

Stores a maintenance or inspection report and returns the hash that gets committed on-chain.

Parameter	In	Required	Type
id	path	yes	string

RESPONSES

200 · errors return {"error": CODE, "message": text}

POST /api/v1/incidents/{id}/maintenance-evidence

Assigned technician: anchors fresh, anomaly-free evidence from a live bound device and returns its event id (used for wallet flows).

Parameter	In	Required	Type
id	path	yes	string

RESPONSES

200 · errors return {"error": CODE, "message": text}

GET /api/v1/incidents

Incidents with status, technician, inspector, reports and the derived technician record.

RESPONSES

200 · errors return {"error": CODE, "message": text}

Devices & commands

GET /api/v1/devices

All devices with provenance, profile, capabilities, binding, last authentication result and latest readings.

RESPONSES

200 · errors return {"error": CODE, "message": text}

POST /api/v1/devices

Admin: provisions a device on-chain (provenance fixed) and returns its secret key ONCE.

REQUEST BODY (JSON SCHEMA)

```
{
  "type": "object",
  "required": [
    "id",
    "provenance",
    "profile"
  ],
  "properties": {
    "id": {
      "type": "string"
    },
    "label": {
      "type": "string",
      "maxLength": 80
    },
    "provenance": {
      "enum": [
        "REAL",
        "SIMULATED"
      ]
    },
    "profile": {
      "type": "string",
      "maxLength": 32
    },
    "caps": {
      "type": "integer",
      "minimum": 0
    },
    "assetId": {
      "type": "string"
    }
  }
}
```

RESPONSES

200 · errors return {"error": CODE, "message": text}

GET /api/v1/devices/{id}/telemetry

Recent authenticated samples from one device with flags, anomalies and delayed markers.

Parameter	In	Required	Type
id	path	yes	string

RESPONSES

200 · errors return {"error": CODE, "message": text}

POST /api/v1/devices/{id}/bind

Admin: binds a device to an asset (new on-chain binding version; closes sessions; invalidates open challenges).

Parameter	In	Required	Type
id	path	yes	string

RESPONSES

200 · errors return {"error": CODE, "message": text}

POST /api/v1/devices/{id}/revoke

Admin: revokes a device on-chain; its evidence is rejected from then on.

Parameter	In	Required	Type
id	path	yes	string

RESPONSES

200 · errors return {"error": CODE, "message": text}

POST /api/v1/devices/{id}/rotate-key

Admin: rotates the device key (on-chain key version) and returns the new secret ONCE.

Parameter	In	Required	Type
id	path	yes	string

RESPONSES

200 · errors return {"error": CODE, "message": text}

GET /api/v1/commands

Actuator command log: created → delivered → acknowledged / rejected / expired / uncertain.

RESPONSES

200 · errors return {"error": CODE, "message": text}

POST /api/v1/commands

Creates a signed, 60-second actuator command (GATE_OPEN custodian-only; PUMP_ON needs on-chain NORMAL, 1–30 s; PUMP_OFF always allowed).

REQUEST BODY (JSON SCHEMA)

```
{
  "type": "object",
  "required": [
    "deviceId",
    "action"
  ],
  "properties": {
    "deviceId": {
      "type": "string"
    },
    "action": {
      "type": "string"
    },
    "params": {
      "type": "object"
    }
  }
}
```

RESPONSES

200 · errors return {"error": CODE, "message": text}

Evidence & integrity

GET /api/v1/evidence

Evidence commitments (canonical evidence hashes) with pipeline status and transaction.

RESPONSES

200 · errors return {"error": CODE, "message": text}

GET /api/v1/integrity/{eventId}

Verifies one evidence record: recomputes keccak256 of the canonical bytes and compares with the on-chain commitment (MATCH / MISMATCH / COMMITMENT_ABSENT / UNVERIFIABLE_CHAIN_UNAVAILABLE / SCHEMA_UNSUPPORTED).

Parameter	In	Required	Type
eventId	path	yes	string

RESPONSES

200 · errors return {"error": CODE, "message": text}

POST /api/v1/integrity/sandbox

Creates a sandbox copy of an evidence record for the tamper demonstration and verifies it.

RESPONSES

200 · errors return {"error": CODE, "message": text}

POST /api/v1/integrity/sandbox/{id}/modify

Changes one field of the sandbox copy (optionally also forging its cached hash) and re-verifies against the chain.

Parameter	In	Required	Type
id	path	yes	string

RESPONSES

200 · errors return {"error": CODE, "message": text}

POST /api/v1/integrity/sandbox/{id}/restore

Restores the sandbox copy from the original and re-verifies.

Parameter	In	Required	Type
id	path	yes	string

RESPONSES

200 · errors return {"error": CODE, "message": text}

Credentials & organisations

GET /api/v1/credentials

Technician and inspector credentials with issue/expiry/revocation state.

RESPONSES

200 · errors return {"error": CODE, "message": text}

POST /api/v1/credentials

Issuer/admin: issues an on-chain TECHNICIAN or INSPECTOR credential valid for N days.

REQUEST BODY (JSON SCHEMA)

```
{
  "type": "object",
  "required": [
    "holder",
    "role",
    "days"
  ],
  "properties": {
    "holder": {
      "type": "string",
      "pattern": "^0x[0-9a-fA-F]{40}$"
    },
    "role": {
      "enum": [
        "TECHNICIAN",
        "INSPECTOR"
      ]
    },
    "days": {
      "type": "number",
      "exclusiveMinimum": 0,
      "maximum": 3650
    }
  }
}
```

RESPONSES

200 · errors return {"error": CODE, "message": text}

POST /api/v1/credentials/revoke

Issuer/admin: revokes an on-chain credential.

RESPONSES

200 · errors return {"error": CODE, "message": text}

GET /api/v1/organizations

Organisations and their members.

RESPONSES

200 · errors return {"error": CODE, "message": text}

POST /api/v1/organizations

Admin: creates an organisation.

REQUEST BODY (JSON SCHEMA)

```
{
  "type": "object",
  "required": [
    "name",
    "kind"
  ],
  "properties": {
    "name": {
      "type": "string",
      "minLength": 2,
      "maxLength": 80
    },
    "kind": {
      "type": "string",
      "maxLength": 32
    }
  }
}
```

RESPONSES

200 · errors return {"error": CODE, "message": text}

POST /api/v1/organizations/members

Admin: adds or moves a wallet address into an organisation.

RESPONSES

200 · errors return {"error": CODE, "message": text}

Blockchain

GET /api/v1/chain/manifest

Deployment manifest: chain id, contract addresses, ABI version, deployment block, fingerprint.

RESPONSES

200 · errors return {"error": CODE, "message": text}

GET /api/v1/chain/status

RPC health, latest block, fingerprint, dev-signer state and indexer cursor.

RESPONSES

200 · errors return {"error": CODE, "message": text}

GET /api/v1/chain/txs

Transactions submitted by the backend or tracked for wallets, with receipt status.

RESPONSES

200 · errors return {"error": CODE, "message": text}

GET /api/v1/chain/events

Decoded contract events indexed from the current deployment (optionally filtered by name).

RESPONSES

200 · errors return {"error": CODE, "message": text}

GET /api/v1/chain/tx/{hash}

Transaction detail read from the node: receipt, block, gas, decoded events and related evidence (the local explorer).

Parameter	In	Required	Type
hash	path	yes	string

RESPONSES

200 · errors return {"error": CODE, "message": text}

POST /api/v1/chain/reindex

Admin: resets the indexer cursor and rebuilds the event index from block 0.

RESPONSES

200 · errors return {"error": CODE, "message": text}

POST /api/v1/chain/track

Records and waits for a transaction sent by the signed-in browser wallet, then refreshes projections.

REQUEST BODY (JSON SCHEMA)

```
{
  "type": "object",
  "required": [
    "hash"
  ],
  "properties": {
    "hash": {
      "type": "string",
      "pattern": "^0x[0-9a-fA-F]{64}$"
    }
  }
}
```

RESPONSES

200 · errors return {"error": CODE, "message": text}

Other

GET /api/v1/stream

Server-Sent Events stream of live updates (telemetry, chain events, jobs, commands...) with Last-Event-ID replay and a 5-second heartbeat.

RESPONSES

200 · errors return {"error": CODE, "message": text}

GET /api/v1/overview

Dashboard counts: assets by condition/lifecycle, open incidents, pending transfers, device health, evidence and job status.

RESPONSES

200 · errors return {"error": CODE, "message": text}

GET /api/v1/audit

Audit log of every operator, oracle and device action (newest first).

RESPONSES

200 · errors return {"error": CODE, "message": text}

GET /api/v1/diagnostics

Operational state: network URLs for devices, oracle job queue, sessions, authentication failures.

RESPONSES

200 · errors return {"error": CODE, "message": text}